SyscallGuard: Cost-Based System Call Budgets with Priority-Aware Borrowing on Linux

BCSE303L – OPERATING SYSTEMS

COURSE PROJECT

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering (Cyber Security)

By

Swayam Prakash Yadav – 24BYB0050

Parth Panchal – 24BYB0069

Trinabha Dixit – 24BYB0026



School of Computer Science Engineering and Information Systems

VIT, Vellore

October 2026

ABSTRACT

Linux limits what a process may use through CPU shares, memory limits, I/O bandwidth and process counts, but it treats every system call as free. A single process that calls `fsync()` in a loop, creates and deletes thousands of files in one directory, or forks without end can slow every other program on the machine, even while its CPU and memory use look small. **SyscallGuard** is a user-space supervisor that gives each job a budget of system calls in which every call is charged by what it costs the kernel, so an `fsync()` is charged about 1,500 times as much as a small `read()`.

A job is started under `sg-launch`, which installs a seccomp-BPF filter on the job. Cheap calls pass through the filter inside the kernel at no extra cost; costed calls are sent to the supervisor `sgd` through seccomp user notification. sgd charges the call against a per-job token bucket, whose rate is the job's priority share of a global budget, and against a global bucket, then lets the call continue, holds it in a priority queue with aging, or fails it with `EAGAIN`. A job that is short of tokens can borrow from an idle or lower-priority job through a loan ledger, and repays from its later refills. Process creation follows a per-job fork policy, so a fork storm is contained while a throttled build still completes. Costs are measured on the target machine with eBPF (bpftrace). The system runs on an unmodified Linux kernel (5.5 or later) without a kernel module or root privileges.

The implementation is about 3,868 lines of C and 2,142 lines of Python, tested by 100 unit checks, 25 tests of the Python tools and 51 end-to-end checks. On a 10-CPU WSL2 machine (Linux 6.6), next to a metadata storm that raised a latency-sensitive job's 99th-percentile latency from 122 µs to 10,621 µs, SyscallGuard brought it back to 355 µs, while a cgroup v2 CPU limit made it worse (189,818 µs). The in-kernel fast path adds about 4 ns per call, a costed call's round trip to the supervisor costs about 18 µs, and an unthrottled build is not measurably slower.

Keywords: system calls, seccomp-BPF, seccomp user notification, token bucket, priority scheduling, aging, resource borrowing, eBPF, Linux, performance isolation.

CONTENTS

INTRODUCTION 1

1.1 Objective 1

1.2 Motivation 2

1.3 Background 2

1.4 Problem Statement 3

1.5 Users and Scope 3

PROJECT DESCRIPTION AND REQUIREMENTS 4

2.1 Project Description 4

2.2 Components 4

2.3 Functional Requirements 5

2.4 Non-functional Requirements 5

2.5 Technical Specification 6

SYSTEM DESIGN 7

3.1 Architecture 7

3.2 Launching a Job 7

3.3 Life of a Costed System Call 8

3.4 Cost Model 9

3.5 Budgets: Token Buckets and Priority Shares 10

3.6 Admission Decision 10

3.7 Waiting Queue and Aging 11

3.8 Borrowing and the Loan Ledger 12

3.9 Process Creation: Fork Policies 12

3.10 Design Decisions 13

OPERATING SYSTEM CONCEPTS APPLIED 14

4.1 Course Concepts in the Project 14

4.2 The seccomp-BPF Filter 14
4.3 Seccomp User Notification 15
4.4 Inter-Process Communication: Passing the Listener 15
4.5 Event-Driven Supervisor 16
4.6 Scheduling Concepts in the Budget 16
4.7 Process and Thread Creation 16
4.8 Kernel Instrumentation with eBPF 16
IMPLEMENTATION 18
5.1 Source Layout 18
5.2 Filter Generator 18
5.3 Launcher 19
5.4 Supervisor Event Loop 19
5.5 Decision Path 20
5.6 Token Bucket 21
5.7 Borrowing 21
5.8 Control Interface and Metrics 21
5.9 Cost Calibration with eBPF 22
5.10 Event Log and Analysis 23
TESTING 24
6.1 Test Strategy 24
6.2 Unit Tests 24
6.3 Integration Tests 24
6.4 Test Run 25
DEMONSTRATIONS 27
7.1 D1: Building the Project 27
7.2 D2: Watching System Calls with User Notification 27
7.3 D3: Priority Shares 28

7.4 D4: Inspecting and Re-prioritising a Running System 29
7.5 D5: Containing Process Creation 30
7.6 D6: Borrowing 31
7.7 D7: Calibrating Costs 32
7.8 D8: Running the Evaluation Harness 33
RESULTS AND DISCUSSION 35
8.1 Method 35
8.2 E1: Per-Call Overhead 36
8.3 E2: Build Overhead 36
8.4 E3: Noisy Neighbour 36
8.5 E4: Fork Pressure 39
8.6 E5: Borrowing 40
8.7 E6: Throttled Build 40
8.8 Where the Overhead Goes 41
8.9 Discussion 42
8.10 Advantages and Limitations 43
CONCLUSION 45
REFERENCES 46
APPENDIX A 47
A.1 How to Run the Project 47
A.2 sgd Options 47
A.3 sg-launch Options 48

LIST OF FIGURES

Fig. 3.1 SyscallGuard Architecture 7

Fig. 3.2 Job Launch and Listener Hand-over 8

Fig. 3.3 Life of a Costed System Call 9

Fig. 3.4 Admission Decision for a Costed Call 11

Fig. 6.1 make test: End of the Test Run 26

Fig. 7.1 Building SyscallGuard 27

Fig. 7.2 Logging openat() Calls Through Seccomp User Notification 28

Fig. 7.3 Priority Shares: Two Identical Jobs and the Event-Log Analysis 29

Fig. 7.4 sgctl status Before and After set-prio, and the /metrics Endpoint 30

Fig. 7.5 Fork Policies reject and brief on the Same Fork Loop 31

Fig. 7.6 Burst with Borrowing: Run Time, Loans and Loan Events 32

Fig. 7.7 The Same Burst with Borrowing Disabled (sgd -B) 32

Fig. 7.8 Cost Calibration with bpftrace and the Resulting Cost Table 33

Fig. 7.9 Evaluation Harness Running E1 and E5 34

Fig. 8.1 E3: Victim 99th-Percentile Latency (log scale) 37

Fig. 8.2 E3: Kernel CPU Time Consumed by the Aggressors in 10 s 38

Fig. 8.3 E4: Process Creation Rate (log scale) 39

Fig. 8.4 Stages of a Costed Call of the Victim Running Alone (medians) 41

LIST OF TABLES

Table 1.1 Users of the System 3

Table 2.1 Components and Their Roles 4

Table 2.2 Functional Requirements 5

Table 2.3 Software and Hardware Used 6

Table 3.1 System Call Classes and Costs 9

Table 3.2 Borrowing Rules 12

Table 3.3 Fork Policies 13

Table 4.1 Course Concepts and Where They Are Used 14

Table 5.1 Source Files 18

Table 5.2 sgctl Commands 22

Table 5.3 Seed and Calibrated Costs (units, 1 unit ≈ 1 µs) 22

Table 6.1 Unit Tests (tests/unit_test.c and Python tools) 24

Table 6.2 Selected Integration Tests 25

Table 7.1 D1: Building the Project 27

Table 7.2 D2: Watching System Calls 27

Table 7.3 D3: Two Identical Jobs with Different Priorities 28

Table 7.4 D4: sgctl and Prometheus Metrics 29

Table 7.5 D5: Fork Policies 30

Table 7.6 D6: A Burst Borrows from an Idle Neighbour 31

Table 7.7 D7: Measuring Call Costs with eBPF 32

Table 7.8 D8: Evaluation Harness 33

Table 8.1 Experiments 35

Table 8.2 E1: Cost of One System Call (ns per call) 36

Table 8.3 E2: Unthrottled make -j4 36

Table 8.4 E3: Victim Latency and Aggressor Throughput 37

Table 8.5 E4: Victim Next to 16 Fork Workers....39
Table 8.6 E5: A Fixed Burst With and Without Borrowing....40
Table 8.7 E6: make -j4 at a Quarter of Its Demand....40
Table 8.8 Victim Alone: Cost of an Occasional Costed Call....41
Table 8.9 Victim Next to 16 Fork Workers: Synchronous Wake-up On and Off....42
Table 8.10 SyscallGuard Compared with the Baselines....43
Table A.1 sgd Command-Line Options....47
Table A.2 sg-launch Command-Line Options....48

LIST OF ABBREVIATIONS

Abbreviation	Expansion
ABI	Application Binary Interface
BPF	Berkeley Packet Filter (classic BPF in seccomp filters)
CPU	Central Processing Unit
eBPF	extended BPF (kernel tracing programs, used by bpftrace)
EAGAIN	Error: resource temporarily unavailable, try again
ENOSYS	Error: function not implemented
fd	File descriptor
FIFO	First In, First Out
HTTP	Hypertext Transfer Protocol
I/O	Input/Output
IOPS	I/O operations per second
IPC	Inter-Process Communication
JSON	JavaScript Object Notation
OS	Operating System
PCB	Process Control Block
PID	Process Identifier
p50, p95, p99	50th, 95th and 99th percentile
SCM_RIGHTS	Socket control message that passes file descriptors
TOCTOU	Time Of Check to Time Of Use
WSL2	Windows Subsystem for Linux, version 2

SYMBOLS AND NOTATIONS

Notation	Meaning
R_g	Global budget: cost units admitted per second across all jobs (sgd -r)
w_j	Weight of job j, equal to its priority 1 (lowest) .. 10 (highest)
r_j	Refill rate of job j: $R_g \cdot w_j / \Sigma w$ (or a fixed rate set with sg-launch -r)
H	Burst horizon; a bucket holds at most rate × H (sgd -H, 1 s)
C_j	Capacity of job j's bucket: $r_j \cdot H$
c	Cost of one system call in units (1 unit ≈ 1 µs of kernel time after calibration)
T_{max}	Longest a call may wait before it fails with EAGAIN (sgd -T, 2 s)
q	Aging quantum: waiting q raises effective priority by one level (sgd -a, 500 ms)
β	Borrowing cap: a job may owe at most $\beta \cdot C_j$ (50%)
k	Minimum loan size as a share of the borrower's capacity (sgd -k, 10%)
J	Jain's fairness index, $(\Sigma x)^2 / (n \cdot \Sigma x^2)$; 1.0 = perfectly proportional
ALLOW / WAIT / REJECT	The three decisions sgd makes for a costed call
µs, ms	Microsecond, millisecond

CHAPTER 1

INTRODUCTION

Every request a program makes to the operating system, such as reading a file, creating a process or flushing data to disk, is a system call. The kernel does very different amounts of work for different calls. Reading a few bytes that are already in the page cache takes about a microsecond; `fsync()` waits for the storage device and took about 1.5 ms on the test machine; creating a process with `fork()` copies page tables and took about 180 µs. Creating and deleting files in one directory takes the directory's lock, so other programs using the same directory have to wait.

Linux already lets an administrator limit CPU time, memory, I/O bandwidth and the number of processes of a group of programs through control groups (cgroups) and resource limits. None of these mechanisms looks at system calls. A program that is well inside its CPU and memory limits can still issue thousands of expensive calls per second and slow down every other program that needs the same kernel resources. This project builds **SyscallGuard**, a tool that gives each job a budget of system calls in which each call is charged by its cost, and that enforces the budget on an unmodified Linux kernel.

1.1 Objective

The objective is to design and implement a system-call governor for Linux that protects well-behaved programs from noisy neighbours, and to measure how well it works and what it costs. Specifically, the project aims to:

- intercept the system calls of a job and its whole process tree without modifying the kernel, using seccomp-BPF and seccomp user notification;
- charge each call a cost that reflects the kernel work it causes, with costs measured on the target machine using eBPF;
- enforce a per-job budget derived from the job's priority with token buckets, and decide for every costed call whether to admit it, hold it or fail it;
- prevent starvation of waiting calls with aging, and let a job with a burst of work borrow unused budget from idle or lower-priority jobs and repay it later;
- contain process creation (fork storms) without breaking normal builds;

- provide tools to inspect and change a running system, and evaluate the design against no control, cgroup v2 and a uniform-cost limiter.

1.2 Motivation

Shared Linux machines, such as build servers, CI runners, university lab servers and container hosts, run programs from many users at once. A common complaint on such machines is that one job makes the whole machine slow even though the CPU is not fully used. Typical causes are a logger that calls `fsync()` after every line, a test suite that creates and deletes thousands of temporary files, a package manager that runs `stat()` on every file in a tree, or a script that forks a new process for every item it handles.

These workloads are expensive because of the kind of system calls they make, not because of the CPU time they consume in user space. CPU limits do not help: when a cgroup's CPU quota runs out, the kernel stops the job's threads wherever they are, possibly while they hold a directory lock, and the other jobs then wait for that lock even longer. A limit on the number of calls per second does not help either, because it treats a one-byte `read()` and an `fsync()` as the same. What is needed is control at the system-call boundary that knows what each call costs, applied before the call enters the kernel.

1.3 Background

System calls and seccomp. A user program enters the kernel through a system call, identified by a number and up to six register arguments. Linux's secure computing mode (seccomp) lets a process attach a filter, written in classic BPF, that runs on every system call the process makes. The filter sees the call number, the architecture and the arguments, and returns an action: allow the call, fail it with an error number, kill the process, or (since Linux 5.0) notify a user-space supervisor. A filter is inherited by every child process and thread and cannot be removed.

Seccomp user notification. When the filter returns `SECCOMP_RET_USER_NOTIF`, the calling thread is put to sleep and a notification is queued on a *listener* file descriptor. A supervisor process reads the notification with the `SECCOMP_IOCTL_NOTIF_RECV` ioctl and answers it with `SECCOMP_IOCTL_NOTIF_SEND`, either with an error number or, since Linux 5.5, with the `CONTINUE` flag, which lets the original call run in the kernel as if nothing had happened. Until the supervisor answers, the thread stays asleep. This is

exactly the hook a budget enforcer needs: it can hold a call for as long as it wants, and it does not need to perform the call itself.

Token buckets. A token bucket is the standard way of enforcing a rate with bursts. Tokens are added at a fixed rate up to a capacity, and an operation is allowed only if it can take its cost in tokens out of the bucket. Over a long period the bucket admits at most the refill rate; over a short period it admits up to its capacity at once.

Priority scheduling and aging. In CPU scheduling, strict priority can starve low-priority work. The classic remedy is aging: the longer a request waits, the higher its effective priority becomes, until it is eventually served. SyscallGuard applies the same idea to waiting system calls.

1.4 Problem Statement

Design and implement a mechanism for Linux that limits how much kernel work each job can cause through system calls, where each call is charged by its measured cost and each job's share follows its priority; that holds over-budget calls instead of failing them wherever possible, prevents starvation, lets jobs lend unused budget to each other, contains process-creation storms, and runs on an unmodified kernel without root privileges, with an overhead small enough that programs that stay within their budget are not slowed down.

1.5 Users and Scope

Table 1.1 Users of the System

User	What they do with the system
Developer on a shared machine	Runs a build, a test suite or a script under `sg-launch` with a priority, so it neither suffers from nor causes noisy-neighbour slowdowns
System administrator	Runs `sgd` with a global budget, calibrates costs for the machine, watches jobs with `sgctl` and Prometheus metrics, changes priorities at run time
Researcher / student	Uses the event log, `sg_analyze.py` and the evaluation harness to study system-call costs, fairness and isolation

The scope is a single Linux machine. SyscallGuard governs the jobs started under `sg-launch`; programs started outside it are not affected. It runs on x86-64 and AArch64 Linux 5.5 or later, uses features of 5.19 and 6.6 when available, and was built and evaluated on Ubuntu 24.04 under WSL2 with Linux 6.6. Limiting memory, CPU time or network bandwidth is out of scope; those are left to cgroups.

CHAPTER 2

PROJECT DESCRIPTION AND REQUIREMENTS

2.1 Project Description

SyscallGuard consists of a supervisor daemon, a job launcher, a control tool and three Python tools. The supervisor `sgd` holds the global budget and one budget per job. A job is any command started with `sg-launch`; the launcher attaches the SyscallGuard filter to itself, hands the filter's listener to sgd, and then executes the command, so the command and everything it starts form one job with one budget. While jobs run, `sgctl` shows their state and changes their priority or rate, sgd serves Prometheus metrics over HTTP, and every decision can be written to a JSON-lines event log for later analysis.

2.2 Components

Table 2.1 Components and Their Roles

Component	Type	Role
`sgd`	C daemon	Receives notifications for costed calls from all jobs and decides ALLOW, WAIT or REJECT; keeps token buckets, the wait queue, the loan ledger, statistics and the event log
`sg-launch`	C program	Installs the seccomp filter, passes the listener fd to sgd with the job's priority, rate, name and fork policy, then executes the command
`sgctl`	C program	Control client: `status`, `loans`, `metrics`, `set-prio`, `set-rate`
`step1_hello`	C program	Stand-alone demonstration of seccomp user notification: logs every `openat()` of a command
`sg_calibrate.py`	Python + bpftrace	Measures the median kernel time of each costed call with eBPF and writes a cost file
`sg_analyze.py`	Python	Turns the event log into wait-time percentiles, rejection rates, utilisation, loan statistics and Jain's fairness index
`sg_wakeup_trace.py`	Python + bpftrace	Splits the latency of a costed call into stages (filter, waking sgd, decision, reply, real call)
`eval/`	C workloads + Python harness	Six experiments and wake-up diagnostics against four baselines

2.3 Functional Requirements

Table 2.2 Functional Requirements

ID	Requirement	Where it is met
FR1	Run any command as a governed job with a priority 1–10, optionally a fixed rate and a name	`sg-launch -p -r -n`
FR2	Govern the whole process tree of a job, including threads and child processes	seccomp filter inheritance (3.2)
FR3	Charge each costed call by its class, its measured cost and, for I/O and memory calls, its size	cost table (3.4, 5.9)
FR4	Give each job a share of the global budget proportional to its priority, with bursts	token buckets (3.5)
FR5	Hold over-budget calls and admit them in priority order when budget returns; fail a call that waited longer than T_{max}	wait heap (3.6, 3.7)
FR6	Prevent starvation of low-priority waiters	aging (3.7)
FR7	Let a job borrow unused budget from idle or lower-priority jobs and repay it from later refills	borrowing, loan ledger (3.8)
FR8	Contain process creation with a per-job policy: brief wait, wait, or reject	`sg-launch -f` (3.9)
FR9	Show jobs, budgets and loans of a running supervisor and change priority or rate at run time	`sgctl` (5.8)
FR10	Export metrics and an event log of every decision	`sgd -m`, `sgd -l` (5.8, 5.10)
FR11	Measure call costs on the target machine	`sg_calibrate.py` (5.9)

2.4 Non-functional Requirements

- **No kernel changes and no root:** run on a stock Linux kernel; enforcement must work as an ordinary user. Only calibration and tracing need root.
- **Low overhead:** calls that are not costed must not leave the kernel; costed calls of a job within budget must not slow it down measurably.
- **Safety:** a job must not be able to escape its budget (for example through another ABI, `clone3` or `io_uring`), and a failure of sgd must not hang jobs forever.
- **Fairness:** service must be proportional to priority, and no waiting call may starve.
- **Conservation:** borrowing moves tokens between jobs and never creates them, so the global budget is never exceeded.
- **Observability:** every decision must be explainable from the event log.

2.5 Technical Specification

Table 2.3 Software and Hardware Used

Component	Technology	Version
Operating system	Ubuntu on WSL2	Ubuntu 24.04.5 LTS
Kernel	Linux (Microsoft WSL2 build)	6.6.114.1-microsoft-standard-WSL2
Language (core)	C11, compiled with GCC, `-O2 -Wall -Wextra -Wpedantic`	13.3.0
C library	GNU C Library	2.39
Build	GNU Make	4.3
Tools and harness	Python (standard library only)	3.12.3
Kernel tracing	bpftrace (eBPF)	0.20.2
Kernel interfaces	seccomp-BPF, seccomp user notification, epoll, timerfd, signalfd, Unix sockets	Linux ≥ 5.5
Processor	13th Gen Intel Core i7-13650HX	10 CPUs given to WSL2
Memory	RAM available to WSL2	7.8 GiB

CHAPTER 3

SYSTEM DESIGN

This chapter describes how SyscallGuard is organised, how a job is set up, what happens on every system call, and the policies sgd applies: the cost model, the budgets, the wait queue with aging, borrowing and process creation.

3.1 Architecture

Fig. 3.1 shows the components at run time. Each job has its own seccomp filter, attached by its `sg-launch` and inherited by every process and thread of the job. The filter answers cheap calls itself and turns costed calls into notifications on the job's listener. sgd holds the listeners of all jobs in one epoll set, so the listener a notification arrives on identifies the job directly, with no lookup by process ID.

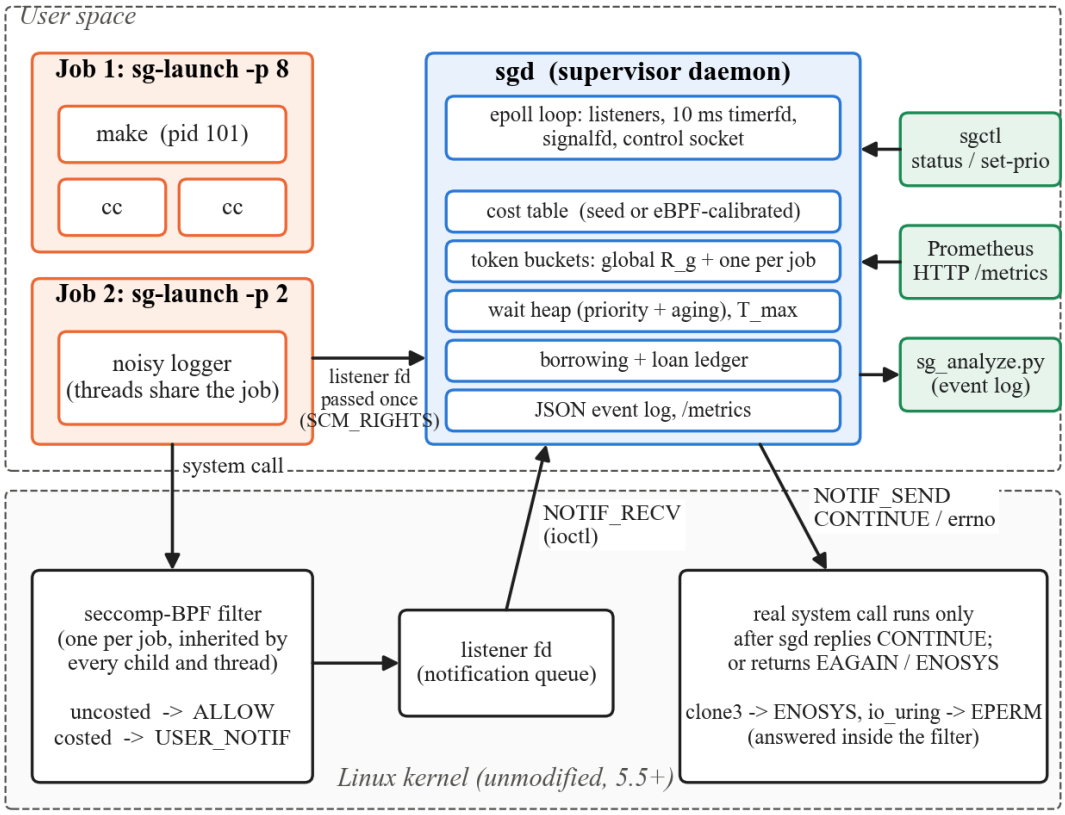


Fig. 3.1 SyscallGuard Architecture

3.2 Launching a Job

A job is set up in the order shown in Fig. 3.2. The order matters. As soon as the filter is installed, any costed call of the launcher blocks until somebody reads the listener, and

the only reader is sgd, which does not have the listener yet. The launcher therefore uses only calls that are not costed (`sendmsg` and `close`) until sgd holds the listener, and its first costed call is the `read` of sgd's acknowledgement, which sgd itself answers. `PR_SET_NO_NEW_PRIVS` is required so that an unprivileged process may install a filter, and it also stops the job from gaining privileges through set-user-ID programs.

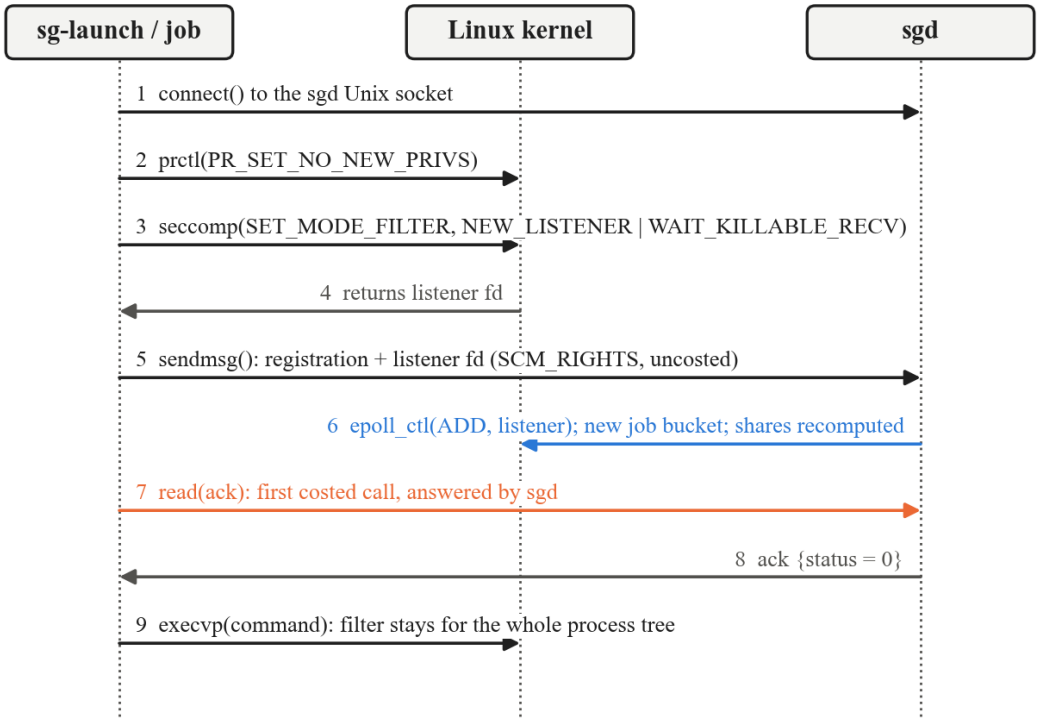


Fig. 3.2 Job Launch and Listener Hand-over

3.3 Life of a Costed System Call

Fig. 3.3 follows one call. Calls that are not in the cost table, such as `getpid()` or `clock_gettime()`, are allowed by the filter inside the kernel and never reach sgd. A costed call such as `openat()` puts the thread to sleep and wakes sgd, which computes the call's cost, refills the buckets and decides. If the call is admitted, sgd replies `CONTINUE` and the kernel executes the original call. If it must wait, sgd keeps the notification and replies later; the thread sleeps in the kernel the whole time and uses no CPU.

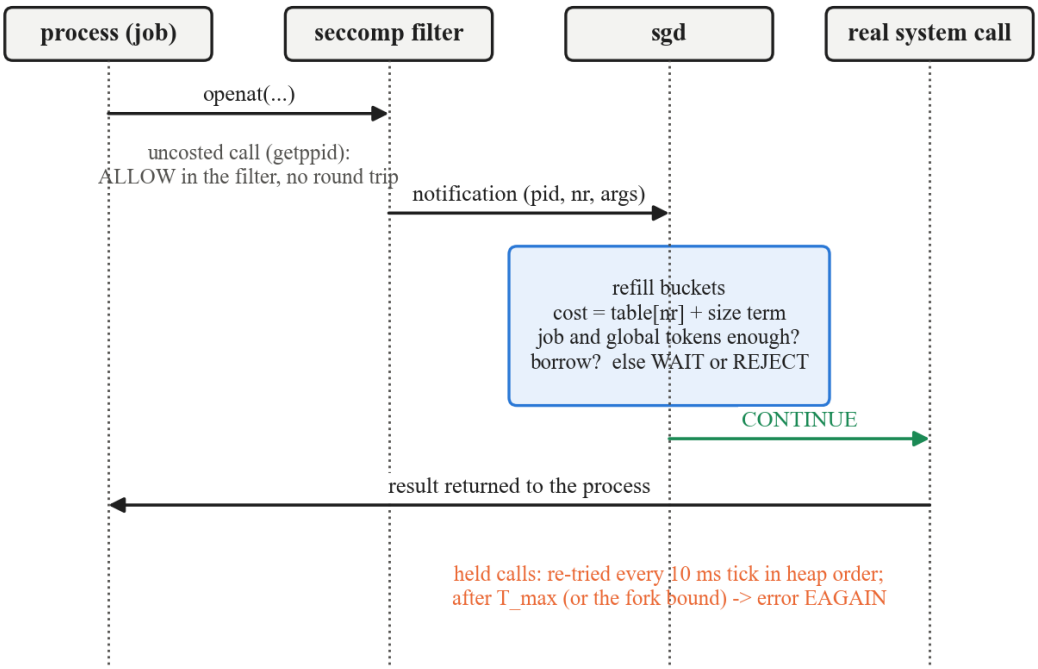


Fig. 3.3 Life of a Costed System Call

3.4 Cost Model

Costed calls are grouped into eight classes (Table 3.1). Each class has a seed cost that only ranks the classes; calibration replaces it with the measured median kernel time of each call on the actual machine, with 1 unit ≈ 1 µs. Data and memory calls also pay a size term, so a 1 MiB `write()` costs more than a 1-byte one. Thread creation is priced separately from process creation by reading the `CLONE_THREAD` flag in `clone()`'s first argument.

Table 3.1 System Call Classes and Costs

Class	System calls	Seed	Size term	Over budget
data_io	read, write, pread64, pwrite64, readv, writev, preadv, pwritev, sendto, recvfrom	1	+1 per 64 KiB	wait
metadata	openat, openat2, open, creat, unlinkat, unlink, rmdir, rename(at/at2), mkdir(at)	3	–	wait
durability	fsync, fdatasync, sync, syncfs	20	–	wait
memory_map	mmap, munmap, mremap, brk	2	+1 per MiB	wait
process	fork, vfork, clone (without CLONE_THREAD)	30	–	fork policy (3.9)
thread	clone with CLONE_THREAD	10	–	wait
exec	execve, execveat	40	–	wait

Class	System calls	Seed	Size term	Over budget
net_setup	socket, connect, bind, accept, accept4	4	–	wait

Two calls are answered inside the filter without a round trip. `clone3()` returns `ENOSYS`, because it passes its flags in memory, where a classic-BPF filter cannot read them; glibc then falls back to `clone()`, whose flags are in a register. `io_uring_setup()` returns `EPERM`, because one `io_uring_enter()` can submit many operations and would bypass per-call accounting. All other calls, about 300 of them, pass straight through.

3.5 Budgets: Token Buckets and Priority Shares

sgd keeps one global bucket with rate R_g and one bucket per job. A costed call of cost c is admitted only if both the job's bucket and the global bucket hold at least c tokens, and c is then taken from both. The global bucket caps the total kernel work of all jobs; the job buckets divide it by priority. A job j with priority w_j receives

$$r_j = R_g \cdot w_j / \Sigma_i w_i \qquad C_j = r_j \cdot H$$

where the sum runs over all jobs that use a share (a job started with a fixed rate is outside the share). Shares are recomputed when a job starts or ends and when its priority changes, so the budget of a job that exits is redistributed at once. Buckets are refilled from the elapsed time whenever they are consulted and on a 10 ms timer, and tokens are kept in thousandths of a unit so that slow rates do not round down to zero.

3.6 Admission Decision

Fig. 3.4 shows the decision for a new notification and for held calls on every timer tick. A call is admitted at once only if its job has no calls already waiting, so a job's calls are admitted in order. A call that can never be admitted, because its cost exceeds what the job's bucket can ever hold (plus the borrowing cap) or the global capacity, is failed at once instead of waiting until T_{max}. Each job may have at most 256 calls waiting.

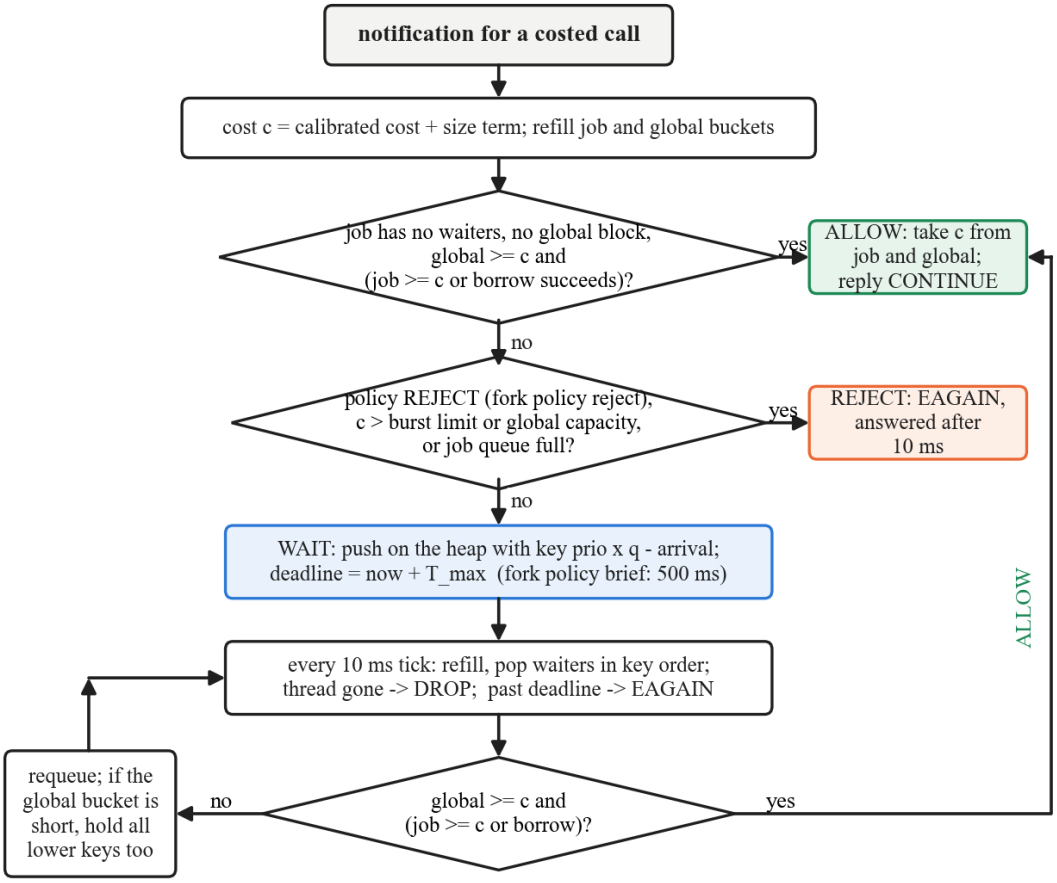


Fig. 3.4 Admission Decision for a Costed Call

Rejections caused by the budget are answered after 10 ms rather than immediately. A program that retries a failed `fork()` in a tight loop would otherwise keep sgd busy answering it; with the delay, the retrying thread sleeps in the kernel instead.

3.7 Waiting Queue and Aging

Held calls are kept in one binary max-heap ordered by effective priority, with ties broken by arrival order. With base priority p, arrival time t_a and aging quantum q, the effective priority at time t is $p + (t - t_a) / q$. Every waiter ages at the same rate, so the term t / q is the same for all of them and the order of two waiters never changes over time. sgd therefore keys the heap on the constant

$$\text{key} = p \cdot q - t_a$$

which gives exactly the same order as the effective priority at any moment, without ever re-sorting the heap. A low-priority call that has waited long enough overtakes newer high-priority calls: with q = 500 ms, a priority-2 call that has waited 3 s ranks

above a priority-7 call that has just arrived. When the call at the head of the heap is held back by the global bucket, the calls behind it are held too, so that small low-priority calls cannot use up the global budget ahead of a large high-priority call.

3.8 Borrowing and the Loan Ledger

A job with a burst of work can run out of tokens while another job's bucket is full and unused. Before making a call wait, sgd tries to borrow the missing tokens, under the rules in Table 3.2. Tokens move from the lender to the borrower, so none are created and the global budget still holds. The ledger records each loan; half of every later refill of the borrower repays its oldest loan. When either side exits, its loans are written off.

Table 3.2 Borrowing Rules

Rule	Purpose
The lender has no waiting calls and owes nothing itself	A job never lends what it needs, and loans do not form chains
The lender keeps a reserve of 30% of its capacity	The lender can still serve its own next calls at once
The borrower may owe at most $\beta = 50\%$ of its capacity	Borrowing absorbs a burst; it does not raise a job's long-term rate
Lenders have equal or lower priority, unless idle for 1 s	Avoids priority inversion through the budget; idle budget is shared with everyone
The eligible lender with the largest surplus wins; ties go to the lower priority	Spreads loans over the jobs that can best afford them
A loan is at least $k = 10\%$ of the borrower's capacity	A throttled job takes a few large loans instead of one per call
Borrow only if the global bucket could then admit the call	A loan that cannot be used would only take tokens from the lender

Because loans are repaid from later refills, borrowing shortens a throttled burst by at most $\beta \cdot C_j$, which is 0.5 s of the job's own rate with the defaults. Section 8.6 measures this bound.

3.9 Process Creation: Fork Policies

Process creation needs special handling. Making `fork()` wait like other calls lets a fork storm build a long queue of sleeping processes, while failing it at once with `EAGAIN` breaks programs such as `make` that treat a failed `fork()` as fatal. Each job therefore chooses a fork policy at launch (Table 3.3). Thread creation always waits, because many programs abort when `pthread_create()` fails.

Table 3.3 Fork Policies

Policy	Behaviour when the job is short of budget	Suited to
brief (default)	Wait for budget up to 500 ms (`sgd -F`), then fail with EAGAIN	General use: a throttled build slows down, a fork storm still fails
wait	Wait like any other call, up to T_{max}	Trusted jobs that must not see EAGAIN
reject	Fail with EAGAIN at once	Untrusted jobs; strictest containment

3.10 Design Decisions

- **Job = listener.** One filter and listener per job means the fd a notification arrives on identifies the job, removing a lookup and a race between a new process's first call and its registration.
- **Decide on registers only.** sgd reads the call number and register arguments from the notification and never dereferences user memory, so the time-of-check to time-of-use problem of seccomp supervisors does not arise: the kernel runs the call with the same registers.
- **Hold before the kernel.** A held call has not entered the kernel and holds no kernel locks, unlike a thread stopped by a CPU quota.
- **Charge before the call.** The cost is charged when the call is admitted, even if the call then fails (for example `execve()` during a PATH search).
- **Fail safe.** If sgd exits, the kernel answers further notifications of its jobs with `ENOSYS`; no job hangs.
- **Single-threaded supervisor.** All state is owned by one thread with one epoll loop, so no locks are needed and every decision is deterministic for a given order of events.

CHAPTER 4

OPERATING SYSTEM CONCEPTS APPLIED

SyscallGuard is built from operating-system mechanisms covered in the course. Table 4.1 maps each concept to its use in the project, and the sections that follow describe the less familiar ones.

4.1 Course Concepts in the Project

Table 4.1 Course Concepts and Where They Are Used

OS concept	Use in SyscallGuard
System calls, user and kernel mode	The unit of control; the filter runs in kernel mode on every call
Process creation (fork, exec), process tree	A job is a process tree; filters are inherited across fork and exec; fork policies
Threads	Threads of a job share its filter and budget; thread creation is priced separately
Process control block	sgd keeps a per-process record (calls, waits, rejections, charged cost) for each PID
CPU scheduling: priority, aging, starvation	Priority shares of the budget; heap ordered by effective priority with aging
Priority inversion	Lending only to equal or higher priority; global-bucket blocking keeps a high-priority waiter first
Inter-process communication	Unix domain sockets; passing the listener fd with SCM_RIGHTS; seccomp notifications
Synchronisation, blocking and wake-up	A held call sleeps in the kernel until sgd replies; synchronous wake-up on Linux 6.6
Deadlock avoidance	Launcher ordering: no costed call until sgd holds the listener (3.2)
Event-driven I/O	epoll over all listeners, a timerfd tick, a signalfd and the control sockets
Security and protection	seccomp-BPF, no_new_privs, blocking x32 and foreign-ABI calls, clone3 and io_uring
Resource management	Token buckets, global and per-job budgets, comparison with cgroup v2
Kernel instrumentation	eBPF tracepoints for cost calibration and per-stage latency

4.2 The seccomp-BPF Filter

The filter is a classic-BPF program generated at launch from the cost table. It first checks the architecture, because system call numbers differ between ABIs: a call

through another architecture (for example the 32-bit `int 0x80` entry on x86-64) kills the process, and x32 calls, whose numbers have bit 30 set, return `ENOSYS`; without these checks a job could reach `fsync()` under a number the table does not contain. Classic BPF has no indirect jump, so the filter is a chain of comparisons, one per costed call, all jumping to a single `RET_USER_NOTIF`. The program for this machine has 52 instructions:

```
ld  [4]                 ; seccomp_data.arch
jeq #AUDIT_ARCH_X86_64, +1, 0
ret #KILL_PROCESS
ld  [0]                 ; seccomp_data.nr
jge #0x40000000, 0, +1  ; x32 ABI
ret #ERRNO(ENOSYS)
jeq #435, 0, +1         ; clone3
ret #ERRNO(ENOSYS)
jeq #425, 0, +1         ; io_uring_setup
ret #ERRNO(EPERM)
jeq #0,  +40, 0         ; read     -> notify
jeq #1,  +39, 0         ; write    -> notify
...                     ; one compare per costed call (40 in total)
jeq #43, +1, 0          ; accept   -> notify
ret #ALLOW              ; not costed: no round trip
ret #USER_NOTIF
```

4.3 Seccomp User Notification

sgd uses four operations on a listener: `SECCOMP_IOCTL_NOTIF_RECV` to read a notification (process ID, call number, arguments and a unique ID), `SECCOMP_IOCTL_NOTIF_ID_VALID` to check that a held thread is still alive before admitting it, `SECCOMP_IOCTL_NOTIF_SEND` to answer, and `SECCOMP_IOCTL_NOTIF_SET_FLAGS` to request synchronous wake-up. The launcher sets `SECCOMP_FILTER_FLAG_WAIT_KILLABLE_RECV` (Linux 5.19), so a held call is not interrupted by ordinary signals, which would otherwise make it fail with `EINTR`, while a fatal signal still ends the process. When the last process of a job has exited and been reaped, the kernel reports `POLLHUP` on the listener, which is how sgd learns that a job has ended.

With synchronous wake-up (Linux 6.6), the kernel wakes the supervisor and the notifying thread on the same CPU, so a round trip does not have to wake an idle CPU. Chapter 8 measures its benefit and a case where it hurts.

4.4 Inter-Process Communication: Passing the Listener

The listener is created in the launcher's process, but it must be read by sgd. A file descriptor is only a number in one process's table, so it is passed as an `SCM_RIGHTS` control message on a Unix domain socket: the kernel installs a new descriptor for the

same open file in sgd. The same socket carries the registration record (priority, rate, name, fork policy) and sgd's acknowledgement. The control socket is created with mode 0600 in `/tmp`, so only the user who started sgd can launch jobs under it or control it.

4.5 Event-Driven Supervisor

sgd is one thread blocked in `epoll_wait()` on all its descriptors: one listener per job, the Unix socket for new jobs and `sgctl`, a `timerfd` that fires every 10 ms to refill buckets and retry held calls, a `signalfd` for SIGINT and SIGTERM, and the optional HTTP metrics socket. Handling signals through a descriptor means a signal never interrupts sgd in the middle of a decision. On shutdown sgd releases every held call with `CONTINUE` and sends the delayed rejections, so no job is left waiting.

4.6 Scheduling Concepts in the Budget

The wait queue is a scheduler for system calls. Priority shares correspond to proportional-share scheduling; the heap with aging corresponds to a priority scheduler with aging, and its time-invariant key is what makes aging cheap (Section 3.7). Two forms of priority inversion are handled. First, a high-priority job never lends to a lower-priority one while it is active, since that could leave it short later. Second, when the highest-priority waiter is held only by the global bucket, lower-priority calls are held too, so they cannot drain the global budget in front of it.

4.7 Process and Thread Creation

On Linux, `fork()`, `vfork()` and `pthread_create()` are all implemented with `clone()`; a thread is a clone that shares the address space and has the `CLONE_THREAD` flag. The filter cannot price `clone3()`, so it fails that call, and glibc 2.34 and later fall back to `clone()`. Tests confirm that `pthread_create()` still works and that threads and processes are charged differently. A filter, and therefore the job's budget, is inherited by every child, so a job cannot escape its budget by forking.

4.8 Kernel Instrumentation with eBPF

Calibration uses bpftrace, which compiles a small script into eBPF programs attached to the `raw_syscalls:sys_enter` and `sys_exit` tracepoints. For the processes of the calibration workload, the script records the time from entry to exit of each system call in a histogram per call number. The median of each histogram becomes that call's cost.

The same tracepoints, together with scheduler tracepoints, are used by `sg_wakeup_trace.py` to split a costed call into stages (Section 8.8).

CHAPTER 5

IMPLEMENTATION

The project has about 3,868 lines of C and 2,142 lines of Python. The C code uses only the C library and Linux headers; the Python tools use only the standard library.

5.1 Source Layout

Table 5.1 Source Files

File	Lines	Contents
`src/sgd.c, .h`	1068	Supervisor: event loop, decisions, waiting, priorities, per-process statistics, event log
`src/sgd_ctl.c`	348	Control commands (status, loans, set-prio, set-rate) and Prometheus metrics over HTTP
`src/sg-launch.c`	181	Job launcher: installs the filter, hands the listener to sgd, execs the command
`src/sgctl.c`	119	Control client
`src/costs.c, .h`	341	Cost table, class policies, cost-file loader, seccomp filter generator
`src/bucket.c, .h`	76	Token bucket with elapsed-time refill in milli-units
`src/waitq.c, .h`	103	Binary heap of held notifications
`src/borrow.c, .h`	245	Lender selection, loans, repayment and write-off
`src/proto.c, .h`	145	Registration and control messages; fd passing with SCM_RIGHTS
`src/common.h`	18	Monotonic clock and milli-unit constant
`src/step1_hello.c`	265	Stand-alone seccomp user-notification demo
`tools/sg_calibrate.py`	283	eBPF cost calibration
`tools/sg_analyze.py`	259	Metrics from the event log
`tools/sg_wakeup_trace.py`	377	Per-stage latency of a costed call
`eval/run_eval.py`	733	Evaluation harness (E1–E6)
`eval/diagnose.py`	228	Wake-up diagnostics
`eval/*.c`	540	Workloads: victim, meta_storm, fsync_flood, cheap_io, fork_pressure, bench_syscall
`tests/`	985	Unit, integration and tool tests

5.2 Filter Generator

`sg_build_filter()` in `costs.c` emits the program listed in Section 4.2 from the cost table at launch. It writes the architecture check, the x32 check and the calls answered inside the filter (`clone3`, `io_uring_setup`), then one `JEQ` instruction per costed call

whose true branch jumps over the remaining compares to the final `RET_USER_NOTIF`; a call that matches none of them falls through to `RET_ALLOW`. Because jump offsets in classic BPF are 8 bits, the generator refuses a table of more than 255 calls.

5.3 Launcher

The core of `sg-launch` installs the filter, hands the listener over and waits for sgd's acknowledgement, in the order of Fig. 3.2:

```
/* WAIT_KILLABLE_RECV (5.19+): a held call is not disturbed by non-fatal signals. */
lfd = (int)syscall(SYS_seccomp, SECCOMP_SET_MODE_FILTER,
                   SECCOMP_FILTER_FLAG_NEW_LISTENER |
SECCOMP_FILTER_FLAG_WAIT_KILLABLE_RECV, &prog);
if (lfd < 0 && errno == EINVAL)
    lfd = (int)syscall(SYS_seccomp, SECCOMP_SET_MODE_FILTER,
SECCOMP_FILTER_FLAG_NEW_LISTENER, &prog);
if (lfd < 0) {
    perror("sg-launch: seccomp");
    return 125;
}

/*
 * From here on a costed call blocks until someone reads the listener, so
 * it must reach sgd first (sendmsg and close are not costed). On failure,
 * closing our copy turns costed calls into ENOSYS instead of a hang.
 */
if (sg_send_fd(sock, &reg, sizeof reg, lfd) < 0) {
    close(lfd);
    _exit(125);
}
close(lfd);

/* This read is itself costed; sgd already holds the listener and answers it. */
n = read(sock, &ack, sizeof ack);
if (n != (ssize_t)sizeof ack) {
    fprintf(stderr, "sg-launch: sgd did not accept the job: %s\n",
            n < 0 ? strerror(errno) : "short reply");
    return 125;
}
if (ack.status) {
    fprintf(stderr, "sg-launch: sgd rejected the job: %s\n", strerror(-ack.status));
    return 125;
}
close(sock);

execvp(argv[optind], argv + optind);
fprintf(stderr, "sg-launch: %s: %s\n", argv[optind], strerror(errno));
return errno == ENOENT ? 127 : 126;
```

5.4 Supervisor Event Loop

sgd blocks in `epoll_wait()` and dispatches each ready descriptor by its tag; a job's listener reports `EPOLLIN` for a new notification and `EPOLLHUP` when the job has ended:

```
for (;;) {
    struct epoll_event evs[64];
    int timeout = cfg.spin_us && sg_now_us() - S.last_notify_us < cfg.spin_us ? 0 : -1;
    int n = epoll_wait(S.ep, evs, 64, timeout), i;

    if (n < 0) {
        if (errno == EINTR)
            continue;
```

```
        die("epoll_wait");
    }
    for (i = 0; i < n; i++) {
        void *tag = evs[i].data.ptr;

        switch (*(enum ep_kind *)tag) {
        ...   /* new jobs, sgctl and HTTP connections */
        case EP_TIMER:
            on_tick();
            break;
        case EP_SIGNAL:
            shutdown_all();
            return 0;
        case EP_JOB:
            if (evs[i].events & EPOLLIN)
                on_notify(tag);
            else if (evs[i].events & (EPOLLHUP | EPOLLERR))
                end_job(tag);
            break;
        }
    }
}
```

5.5 Decision Path

`on_notify()` handles one notification. The fast case admits the call at once; otherwise the class policy and the job's fork policy decide between rejecting and holding:

```
    /* Keep per-job FIFO order, and do not overtake a waiter held for the global bucket. */
    if (j->acct.nwaiting == 0 && !(S.g_blocked && w.key <= S.g_blocked_key) &&
        sg_bucket_can(&S.global, w.cost) && job_can(j, w.cost, now)) {
        admit(j, p, &w, 0);
        return;
    }

    policy = sg_classes[cls].policy;
    max_wait_ms = cfg.tmax_ms;
    if (cls == SG_C_PROC) {
        policy = j->fork_policy == SG_FORK_REJECT ? SG_REJECT : SG_WAIT;
        if (j->fork_policy == SG_FORK_BRIEF && cfg.fork_wait_ms < max_wait_ms)
            max_wait_ms = cfg.fork_wait_ms;
    }

    burst_limit = j->acct.bucket.capacity;
    if (cfg.borrow.enabled)
        burst_limit += j->acct.bucket.capacity * cfg.borrow.cap_pct / 100;
    if (policy == SG_REJECT)
        err = sg_classes[cls].reject_errno;
    else if (w.cost > burst_limit || w.cost > S.global.capacity)
        err = EAGAIN;   /* could never be admitted */
    else if (j->acct.nwaiting >= cfg.qmax)
        err = EAGAIN;
    if (err) {
        reject(j, w.id, err, now);
        NOTE(j, p, rejected);
        log_call(j, w.pid, w.nr, w.cls, w.cost, "REJECT", 0);
        return;
    }

    w.seq = S.seq++;
    w.deadline_us = now + max_wait_ms * 1000;
    if (sg_waitq_push(&S.wq, &w) < 0)
        die("sgd: waitq");
    j->acct.nwaiting++;
    NOTE(j, p, waited);
    log_call(j, w.pid, w.nr, w.cls, w.cost, "WAIT", 0);
}
```

5.6 Token Bucket

A bucket stores its last refill time and is refilled from the elapsed time, carrying the sub-milli-unit remainder so that no tokens are lost between frequent refills:

```
int64_t sg_bucket_refill(struct sg_bucket *b, int64_t now_us)
{
    int64_t dt = now_us - b->last_us, acc, gross;

    if (dt <= 0)
        return 0;
    b->last_us = now_us;
    if (dt > MAX_REFILL_US)
        dt = MAX_REFILL_US;

    /* units/s * us / 1000 = milli-units */
    acc = b->rate * dt + b->rem;
    gross = acc / 1000;
    b->tokens += gross;
    b->rem = acc % 1000;
    if (b->tokens >= b->capacity) {
        b->tokens = b->capacity;
        b->rem = 0;
    }
    return gross;
}
```

5.7 Borrowing

`borrow.c` keeps one account per job (its bucket, priority, waiting count, amount owed and amount lent) and a list of open loans per borrower, oldest first. `sg_borrow()` computes the deficit, checks it against the borrower's cap of $\beta \cdot C_j$ minus what it already owes, and rounds the loan up to k = 10% of the borrower's capacity without leaving more than its capacity after the call. `sg_pick_lender()` then scans the accounts and applies the rules of Table 3.2; if no lender can afford the rounded amount, the exact deficit is tried. The tokens are moved, the loan is appended to the borrower's list, and a `loan_open` event is logged. On every refill, `sg_acct_refill()` uses half of the gross refill to repay the oldest loan, crediting the lender up to its capacity, and closes the loan with a `loan_repaid` event once it is paid. `sg_acct_close()` writes off a departing job's loans in both directions.

5.8 Control Interface and Metrics

`sgctl` connects to the same Unix socket as `sg-launch` and sends a control message instead of a registration. Table 5.2 lists the commands. With `sgd -m PORT`, sgd also serves the same metrics in the Prometheus text format on `http://127.0.0.1:PORT/metrics`, from the same epoll loop.

Table 5.2 sgctl Commands

Command	Effect
`sgctl status`	Global bucket, totals, and one row per job: priority, mode, rate, tokens, capacity, waiting, calls, allowed, waited, rejected, owed, lent, fork policy
`sgctl loans`	Outstanding loans: ID, lender, borrower, amount, repaid, age
`sgctl metrics`	Prometheus text (same as the HTTP endpoint)
`sgctl set-prio JOB P`	Change a job's priority; shares are recomputed and its waiters re-keyed
`sgctl set-rate JOB R`	Give a job a fixed rate of R units/s; 0 returns it to a priority share

5.9 Cost Calibration with eBPF

`sg_calibrate.py` starts bpftrace, runs a built-in workload that exercises every costed class for 5 s, and takes the median of each call's latency histogram. Table 5.3 shows the result of a fresh calibration run for this report next to the seed costs and the calibration file in the repository (`costs/wsl2-6.6.conf`). The two calibrations agree within a few per cent for the expensive calls, and they show why seed costs are not enough: `fsync()` costs about 75 times its seed.

Table 5.3 Seed and Calibrated Costs (units, 1 unit ≈ 1 µs)

System call	Class	Seed	Repository file	Fresh run	Samples
`fsync`	durability	20	1,504	1,536	754
`vfork`	process	30	181	189	378
`clone`	process	30	167	180	378
`fdatasync`	durability	20	167	185	754
`execve`	exec	40	112	121	378
`mkdir`	metadata	3	71	82	12,070
`clone_thread`	thread	10	45	46	378
`connect`	net_setup	4	9	10	12,070
`openat`	metadata	3	4	6	25,438
`unlink`	metadata	3	4	8	12,073
`mmap`	memory_map	2	3	4	15,477
`write`	data_io	1	3	3	12,152
`read`	data_io	1	1	1	13,902

5.10 Event Log and Analysis

With `sgd -l FILE`, sgd writes one JSON object per line for every decision (ALLOW, WAIT, REJECT, DROP, with the wait time), for each loan opened, repaid or written off, for each process and job when it ends, and for sgd's start and stop. `sg_analyze.py` reads this log and reports, per job and overall, the admission-wait percentiles, the waited and rejected percentages, the rejection rate per class, budget utilisation (charged / entitled), loan count, volume and repayment time, and Jain's fairness index over charged units per second divided by priority weight:

$$J = (\Sigma_i\, x_i)^2 \,/\, (n \cdot \Sigma_i\, x_i^2), \qquad x_i = \text{charged}_i \,/\, (w_i \cdot \text{lifetime}_i)$$

J = 1.0 means every job received service exactly in proportion to its priority.

CHAPTER 6

TESTING

6.1 Test Strategy

The tests run with `make test` as an ordinary user and need no root. They are in three layers: unit tests of the core data structures in C, tests of the Python tools on recorded inputs, and integration tests that run real programs under `sg-launch` and check sgd's decisions through exit codes, timing, sgd's per-job summaries and the event log.

6.2 Unit Tests

Table 6.1 Unit Tests (tests/unit_test.c and Python tools)

Test	What it checks
`test_bucket`	Refill from elapsed time, capacity clamp, remainder carried between frequent refills
`test_waitq`	Heap order by key and arrival sequence; push/pop of many waiters
`test_aging_key`	The constant key orders waiters exactly like prio + wait / q at any time
`test_costs`	Class lookup, size term, `clone` reclassified as thread with `CLONE_THREAD`, uncosted calls on the fast path, filter ending in ALLOW then USER_NOTIF
`test_calibrated_costs`	Loading, overriding and rejecting cost files
`test_borrow`	Lender choice, reserve, cap, priority rule, idle rule, repayment, write-off on exit, conservation of tokens
`test_borrow_chunks`	Minimum loan size and its limits
`test_analyze.py`, `test_calibrate.py`, `test_wakeup_trace.py`	Percentiles, fairness index and loan statistics; histogram median; per-stage trace parsing

6.3 Integration Tests

`tests/integration_test.sh` starts its own sgd on a private socket for each group and runs 45 checks, of which Table 6.2 lists a selection. `tests/step1_test.sh` runs 6 checks of the stand-alone notification demo.

Table 6.2 Selected Integration Tests

Group	Check
Basics	Command output and exit status are unchanged; a missing command returns 127; `pthread_create` works (clone3 falls back to clone); nothing is rejected under the default budget
Fork policies	reject: forks over budget fail with EAGAIN; brief (default): forks wait for refill and all succeed, or fail after the bound; wait: the same forks wait and succeed
Data I/O over budget	A throttled `dd` still succeeds, is slowed to the budget (12,000 units at 4,000/s take about 2.1 s), and its calls waited instead of failing
Priority shares	A priority-8 job finishes before an identical priority-2 job
Calls that cannot fit	A call larger than the whole burst is rejected at once; a call still waiting after T_{max} is rejected
Borrowing	A call larger than the job's burst succeeds by borrowing from a lower-priority job; the loan is recorded, repaid from later refills and shown by sg_analyze
Lending rules	An active higher-priority job does not lend to a lower one; once idle it lends to anyone; with `-B` the call fails
Calibrated costs	`sgd -L` shows calibrated next to seed costs; an invalid cost file is refused; the committed calibration files load
sgctl and /metrics	status, set-prio, set-rate, loans and metrics work; unknown jobs are errors; HTTP /metrics serves the same text

6.4 Test Run

Fig. 6.1 shows the end of a test run in a terminal. Every check passed: 100 unit checks (reported as one line), 25 Python tool tests, 6 notification-demo checks and 45 integration checks; none failed.

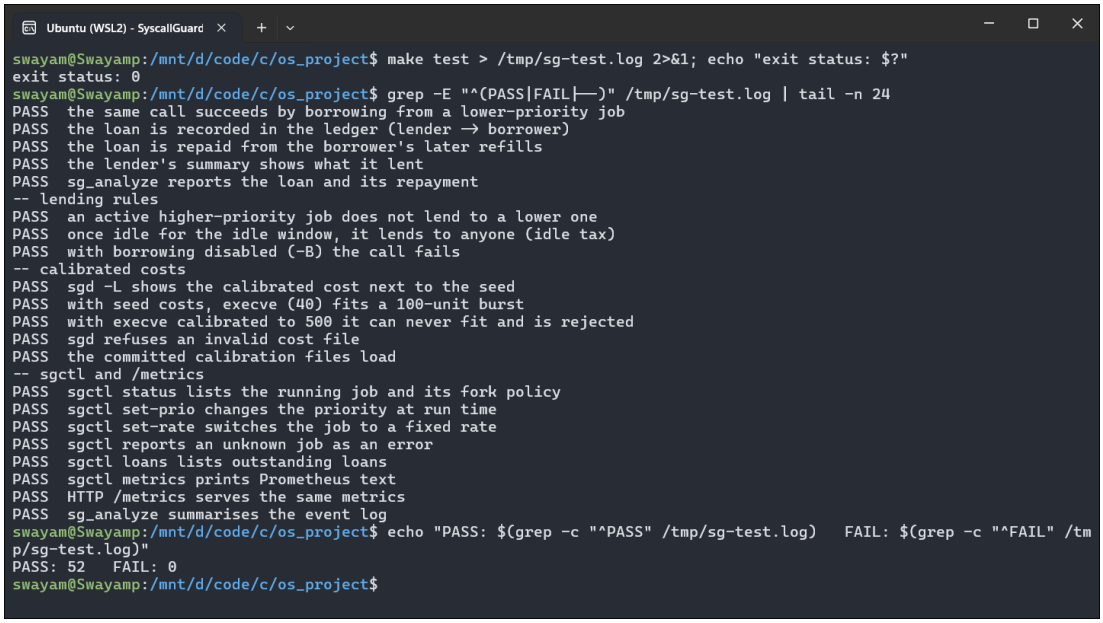


Fig. 6.1 make test: End of the Test Run

CHAPTER 7

DEMONSTRATIONS

This chapter shows SyscallGuard running. Each demonstration was run in a Windows Terminal window connected to Ubuntu under WSL2, and the figures are screen captures of those windows. Each table lists the operation, the commands, the operating-system mechanisms involved and the result.

7.1 D1: Building the Project

Table 7.1 D1: Building the Project

Aspect	Details
Operation	Build sgd, sg-launch, sgctl and step1_hello from a clean tree
Commands	`make clean`, `make`
OS mechanisms	None at run time; compiled with `-std=c11 -O2 -Wall -Wextra -Wpedantic`
Result	Four binaries in `build/`, compiled without warnings

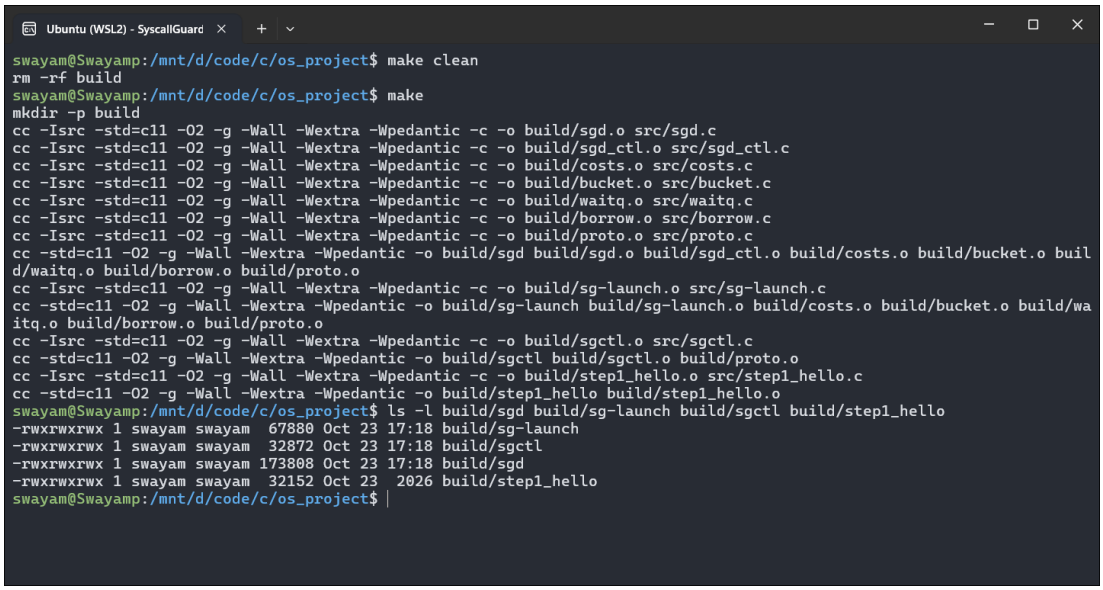


Fig. 7.1 Building SyscallGuard

7.2 D2: Watching System Calls with User Notification

Table 7.2 D2: Watching System Calls

Aspect	Details
Operation	Run `cat` and `ls` under the stand-alone demo, which logs every `openat()` and lets it continue

Aspect	Details
Commands	`./build/step1_hello cat /etc/hostname`, `./build/step1_hello ls /`
OS mechanisms	seccomp filter with `RET_USER_NOTIF` for openat; `NOTIF_RECV`; `NOTIF_SEND` with `CONTINUE`
Result	Each file the program opens (shared libraries, locale files, the target) is printed with its PID; the program's own output is unchanged

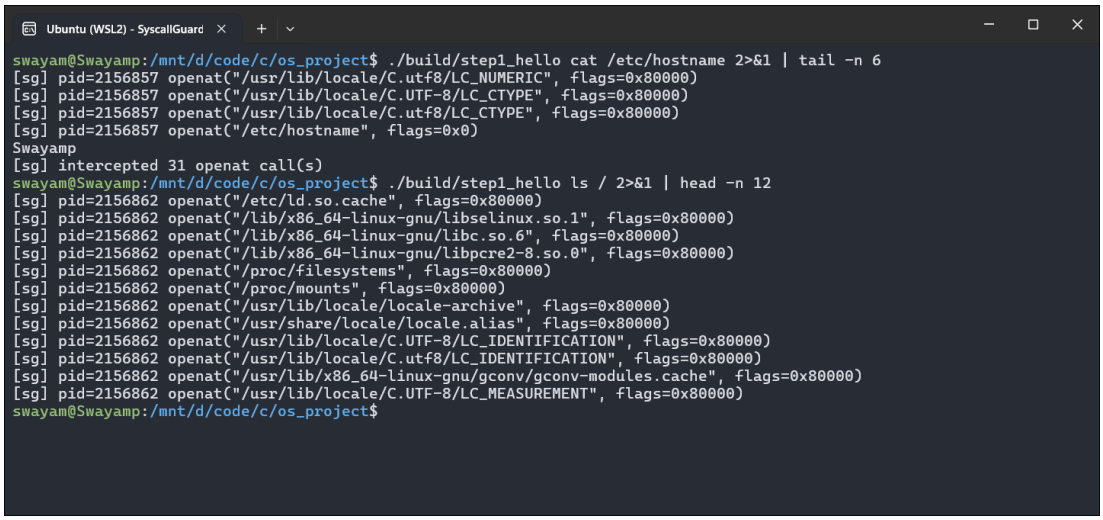


Fig. 7.2 Logging openat() Calls Through Seccomp User Notification

7.3 D3: Priority Shares

Table 7.3 D3: Two Identical Jobs with Different Priorities

Aspect	Details
Operation	Run the same `dd` (2,000 × 64 B, about 4,300 units with seed costs) as a priority-8 job and a priority-2 job under a global budget of 4,000 units/s
Commands	`sgd -r 4000 -l events.jsonl`, `sg-launch -p 8 …`, `sg-launch -p 2 …`, `sg_analyze.py`
OS mechanisms	Token buckets with priority shares (3,200 and 800 units/s); calls held in the kernel while waiting
Result	Both jobs made 4,070 costed calls and none was rejected. The priority-8 job finished in 0.35 s and the priority-2 job in 1.16 s. sg_analyze reports Jain's fairness index over charged units per second per priority weight as 0.991, close to perfectly proportional service.

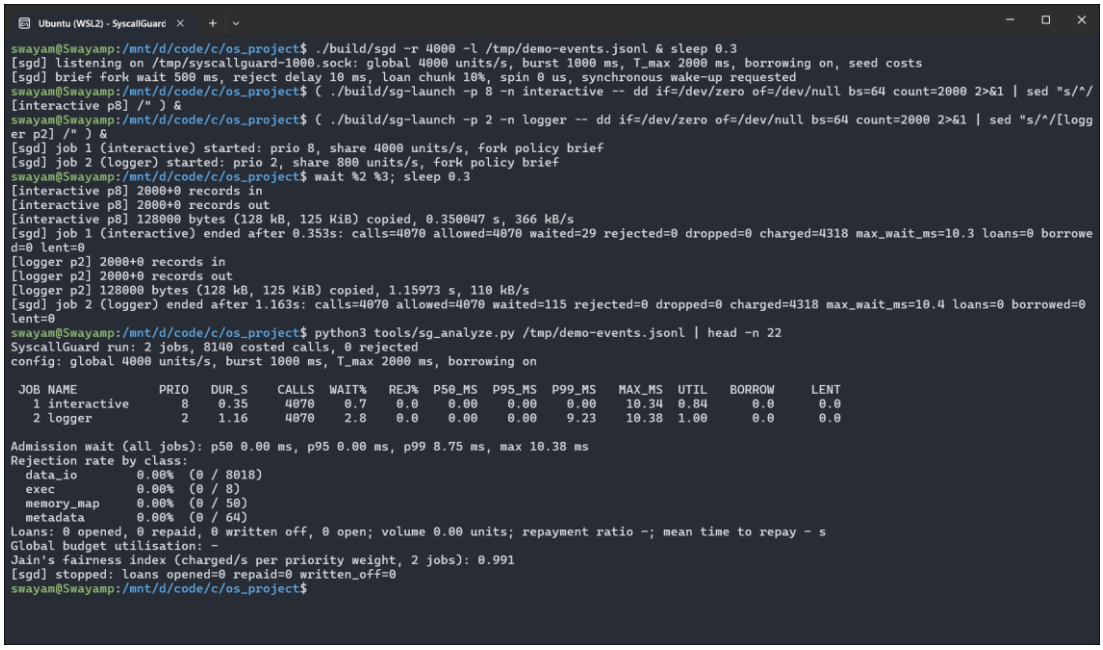


Fig. 7.3 Priority Shares: Two Identical Jobs and the Event-Log Analysis

7.4 D4: Inspecting and Re-prioritising a Running System

Table 7.4 D4: sgctl and Prometheus Metrics

Aspect	Details
Operation	While two long jobs run, show their state, raise the low-priority job to priority 9, and read the metrics endpoint
Commands	`sgctl status`, `sgctl set-prio 2 9`, `curl http://127.0.0.1:9100/metrics`
OS mechanisms	Control messages over the Unix socket; shares recomputed and waiters re-keyed at run time; HTTP served from sgd's epoll loop
Result	Before the change the jobs ran at 2,400 and 600 units/s (8 : 2 of 3,000). After `set-prio 2 9`, the logger ran at 1,588 units/s and the interactive job at 1,411 units/s (9 : 8). The metrics endpoint reports the same priorities and rates.

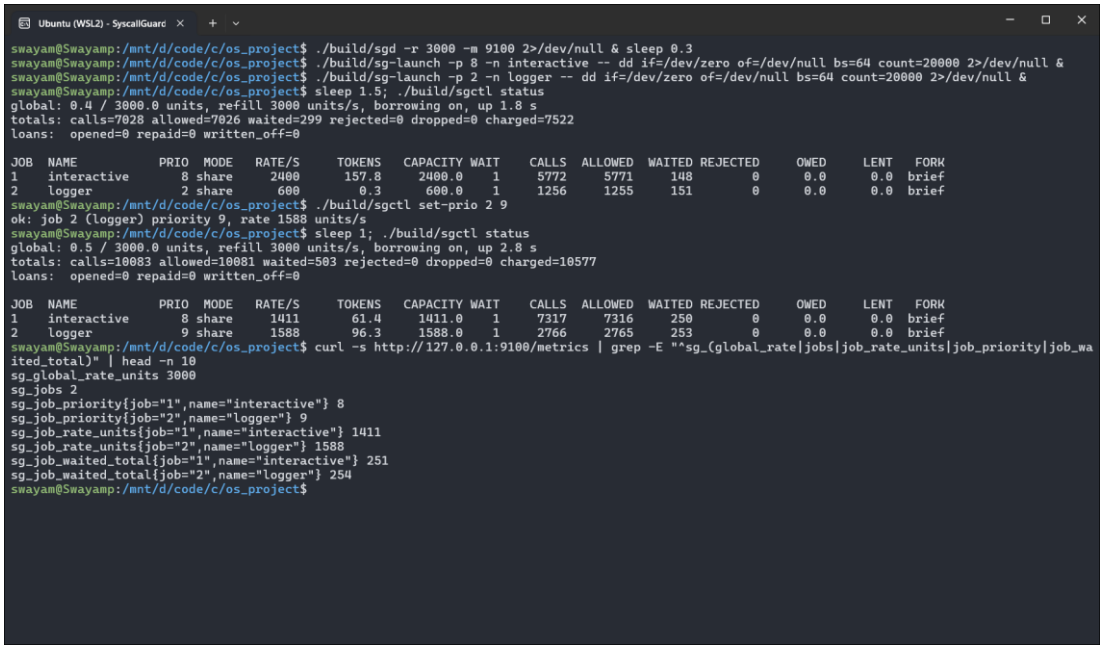


Fig. 7.4 sgctl status Before and After set-prio, and the /metrics Endpoint

7.5 D5: Containing Process Creation

Table 7.5 D5: Fork Policies

Aspect	Details
Operation	Run a program that forks 300 short-lived children, under a 2,000 units/s budget, first with fork policy reject and then with the default policy brief
Commands	`sg-launch -f reject -- ./build/forkn 300`, `sg-launch -- ./build/forkn 300`
OS mechanisms	clone without CLONE_THREAD priced as a process; per-job fork policy; paced rejection
Result	With reject, 199 of the 300 forks succeeded and 101 failed at once with EAGAIN; the run took 2.0 s. With brief, all 300 forks succeeded: 303 calls waited for budget, at most 19.9 ms each, and the run took 4.5 s.

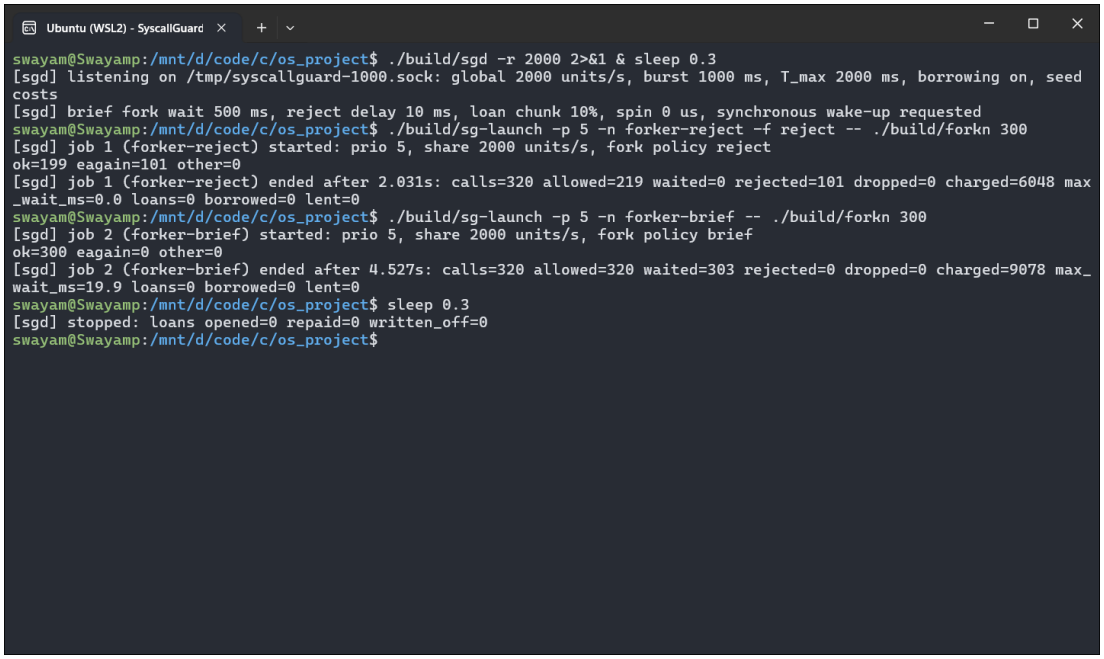


Fig. 7.5 Fork Policies reject and brief on the Same Fork Loop

7.6 D6: Borrowing

Table 7.6 D6: A Burst Borrows from an Idle Neighbour

Aspect	Details
Operation	Run a fixed burst (`dd`, 10,000 × 64 B) at priority 5 next to an idle priority-5 job, with R_g = 20,000 units/s; then repeat with borrowing disabled
Commands	`sgd -r 20000 -l borrow.jsonl`; `sgd -B -r 20000`; `sgctl loans`
OS mechanisms	Loan ledger; tokens moved between buckets; repayment from later refills
Result	With the seed costs used here the burst costs about 20,000 units (1 per read, 1 per write), twice the job's 10,000-unit capacity. With borrowing it finished in 0.54 s after 53 loans of 1,000 units (10% of capacity) from the idle job; with borrowing disabled it took 1.07 s. The saving is about 0.5 s, the bound of Section 3.8.

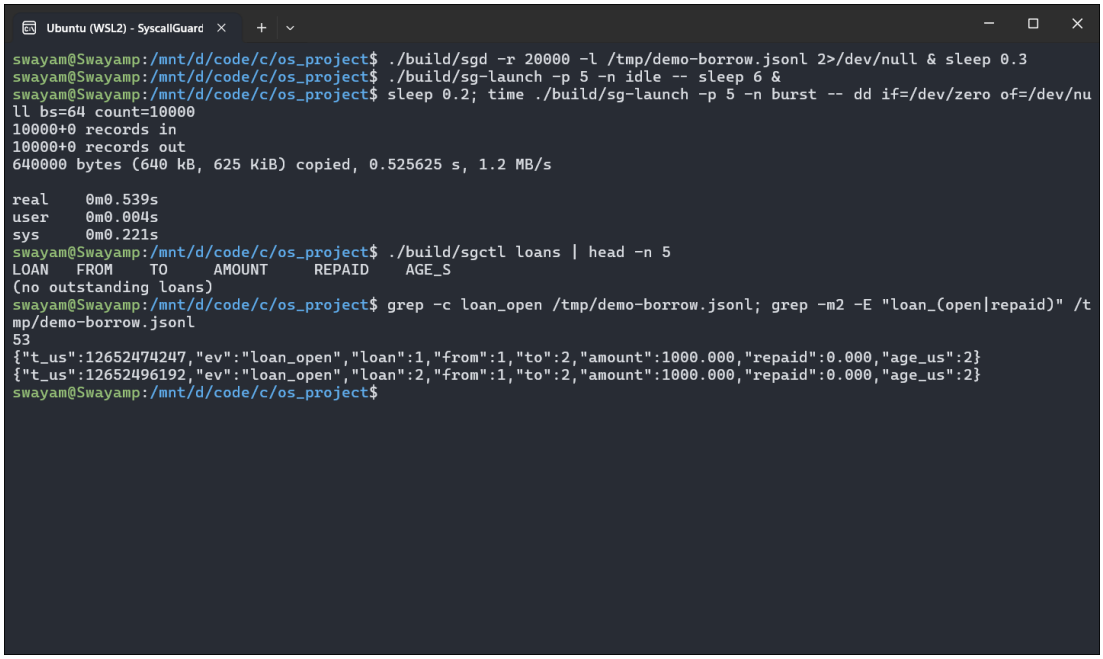


Fig. 7.6 Burst with Borrowing: Run Time, Loans and Loan Events

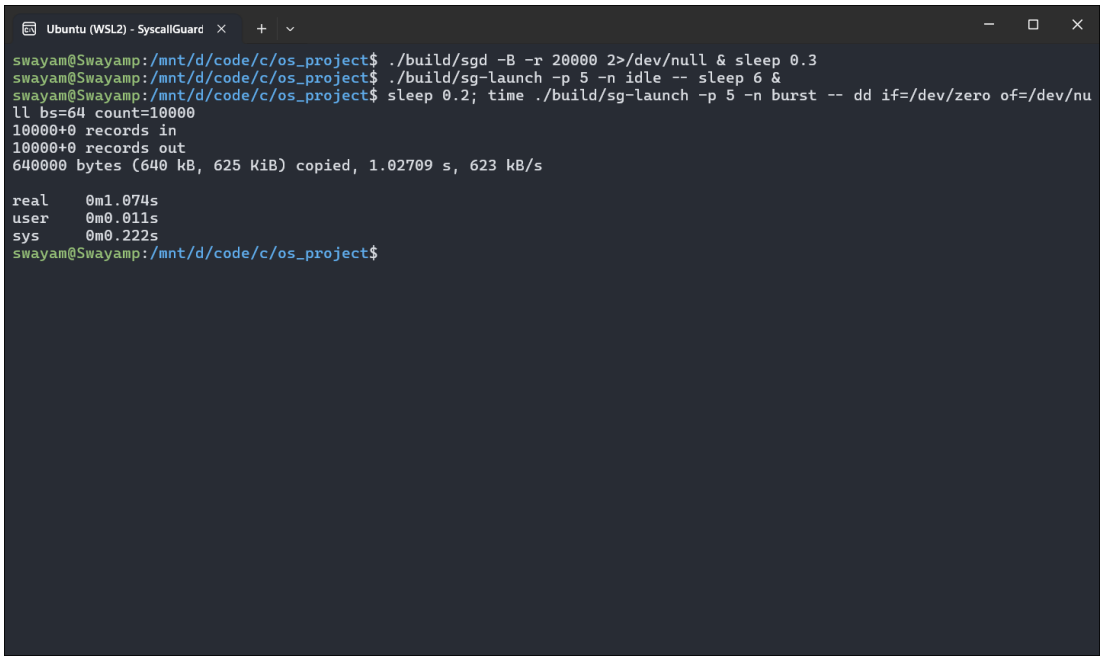


Fig. 7.7 The Same Burst with Borrowing Disabled (sgd -B)

7.7 D7: Calibrating Costs

Table 7.7 D7: Measuring Call Costs with eBPF

Aspect	Details
Operation	Measure the median kernel time of each costed call on this machine and load the result into sgd
Commands	`sudo python3 tools/sg_calibrate.py -o calib.conf`, `sgd -L -c calib.conf`
OS mechanisms	eBPF programs on the raw_syscalls tracepoints (bpftrace), run as root

Aspect	Details
Result	This run measured fsync at 1,527 units, fdatasync 181, vfork 173, clone 165, execve 105, mkdir 78 and clone_thread 33, close to Table 5.3. Calls that take only a few microseconds vary between runs (openat: 1 here, 6 in Table 5.3).

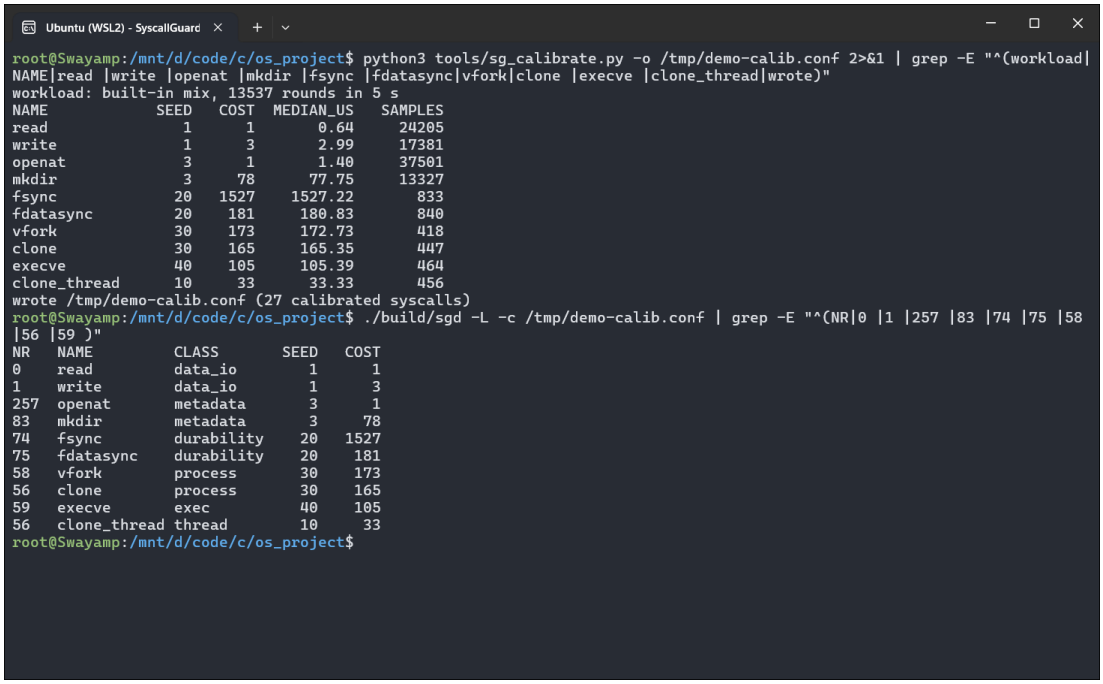


Fig. 7.8 Cost Calibration with bpftrace and the Resulting Cost Table

7.8 D8: Running the Evaluation Harness

Table 7.8 D8: Evaluation Harness

Aspect	Details
Operation	Run experiments E1 (per-call overhead) and E5 (borrowing) once, as a short example of the harness used in Chapter 8
Commands	`python3 eval/run_eval.py --only E1,E5 --reps 1`
OS mechanisms	Workloads started under sg-launch with fresh sgd instances; results collected from sgd's logs and the workloads' JSON output
Result	One repetition gave 74 ns for a native getppid and 79 ns under sgd, 18.8 µs for a costed 1-byte read, and an E5 burst of 3.08 s without borrowing and 2.58 s with it, in line with the full run in Chapter 8.

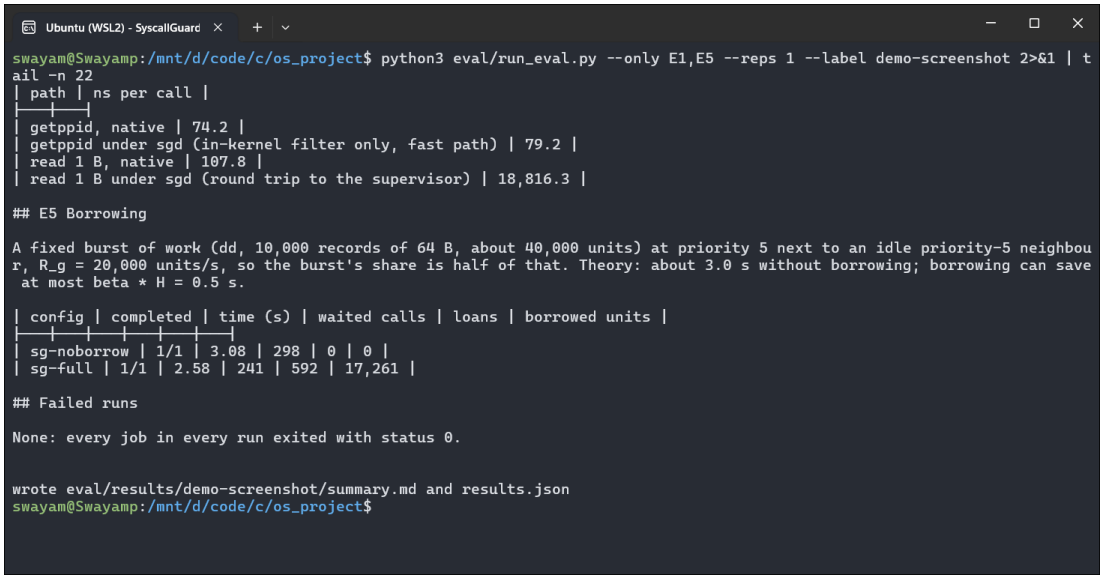


Fig. 7.9 Evaluation Harness Running E1 and E5

CHAPTER 8

RESULTS AND DISCUSSION

SyscallGuard was evaluated against three baselines on six experiments, plus a breakdown of where the time of a costed call goes. All numbers in this chapter come from one fresh run of the evaluation harness made for this report (`eval/results/wsl2-6.6-report`), using the calibration file in the repository.

8.1 Method

- **Machine:** Ubuntu 24.04 under WSL2, Linux 6.6, 10 CPUs of an Intel Core i7-13650HX, 7.8 GiB RAM; the machine was otherwise idle during the runs.
- **Repetitions:** every cell was run 3 times; tables give the mean ± standard deviation. Contention runs last 10 s.
- **Budget:** R_g = 200,000 units/s for E3 and E4, calibrated costs from `costs/wsl2-6.6.conf`, all other sgd options at their defaults.
- **Configurations:** *no control*; *cgroup v2*, with the aggressors in a group limited to one CPU (`cpu.max`), 200 write IOPS (`io.max`) and 32 processes (`pids.max`); *uniform-cost limiter*, which is sgd with every costed call costing 1 and no borrowing, i.e. a plain calls-per-second limiter with the same budget and priorities; *SyscallGuard without borrowing*; and *full SyscallGuard*.
- **Workloads:** purpose-written C programs, all bounded in time, processes and disk space (Table 8.1).

Table 8.1 Experiments

	Workload	Question
E1	`bench_syscall`: getppid (never costed) and 1-byte reads (costed)	What does one call cost natively, on the fast path, and with a round trip to sgd?
E2	`make -j4` of this project, unthrottled	How much does governance slow a real build?
E3	victim (priority 8) next to a metadata storm, an fsync flood and cheap I/O (priority 2)	Is a latency-sensitive job protected from a noisy neighbour, and what does cost weighting add?
E4	victim next to 16 workers forking short-lived children	Is process creation contained, and at what cost?
E5	a fixed `dd` burst next to an idle job	How much does borrowing help a bursty job?

	Workload	Question
E6	`make -j4` throttled to a quarter of its demand	Does a throttled build complete under each fork policy?

8.2 E1: Per-Call Overhead

Table 8.2 E1: Cost of One System Call (ns per call)

Path	ns per call
getppid, native	78.9 ± 0.8
getppid under sgd (in-kernel filter only, fast path)	82.6 ± 3.7
read 1 B, native	102.3 ± 1.6
read 1 B under sgd (round trip to the supervisor)	17,731.7 ± 281.2

A call that is not costed only runs the BPF filter in the kernel and became 4 ns slower. A costed call that has to go to sgd and back costs about 18 µs when made back to back, about 170 times a native 1-byte read. This is why the cost table sends only calls that do real kernel work to sgd: for those, 18 µs is small next to what the call itself costs (an `fsync()` takes 1.5 ms), and calls such as `getpid()` or `clock_gettime()` never leave the kernel.

8.3 E2: Build Overhead

Table 8.3 E2: Unthrottled make -j4

config	build time (s)	slowdown	costed calls/s	sgd CPU (s)
no control	1.16 ± 0.25			
under sgd	1.08 ± 0.04	-6.9%	11,098 ± 375	0.04 ± 0.01

With a budget large enough not to throttle it, the build made about 11,098 ± 375 costed calls per second through sgd and was not measurably slower: the difference is inside the run-to-run variation, and sgd used 0.04 ± 0.01 s of CPU for the whole build.

8.4 E3: Noisy Neighbour

This is the main experiment. A latency-sensitive *victim* (priority 8) opens a file, writes 64 bytes and closes it in a shared directory every 10 ms and records how long each round takes. Next to it, three aggressors at priority 2 run: a metadata storm (16 workers creating and deleting files in the same directory), an fsync flood (4 workers) and a cheap-I/O loop. Table 8.4 gives the full results; Fig. 8.1 and 8.2 show the victim's tail latency and the kernel CPU the aggressors consumed.

Table 8.4 E3: Victim Latency and Aggressor Throughput

Config	Victim p50 (µs)	p95 (µs)	p99 (µs)	Storm ops/s	fsync/s	Aggressor kernel CPU (s)	Rejected
victim alone, no control	62 ± 1	92 ± 1	122 ± 2	–	–	–	–
victim alone, under sgd	322 ± 3	530 ± 20	623 ± 38	–	–	–	0
no control	48 ± 2	7,829 ± 355	10,621 ± 353	76,546 ± 1,373	458 ± 23	65.94 ± 1.18	–
cgroup v2	83 ± 6	127,655 ± 43,328	189,818 ± 870	2,936 ± 199	88 ± 7	9.27 ± 0.01	–
uniform-cost limiter	77 ± 2	204 ± 101	423 ± 216	12,322 ± 199	2,737 ± 740	10.71 ± 0.10	0
SyscallGuard, no borrowing	116 ± 4	399 ± 180	1,014 ± 1,018	3,510 ± 9	26 ± 0	5.90 ± 0.20	0
SyscallGuard (full)	139 ± 14	269 ± 22	355 ± 39	3,517 ± 2	26 ± 0	5.62 ± 0.21	0

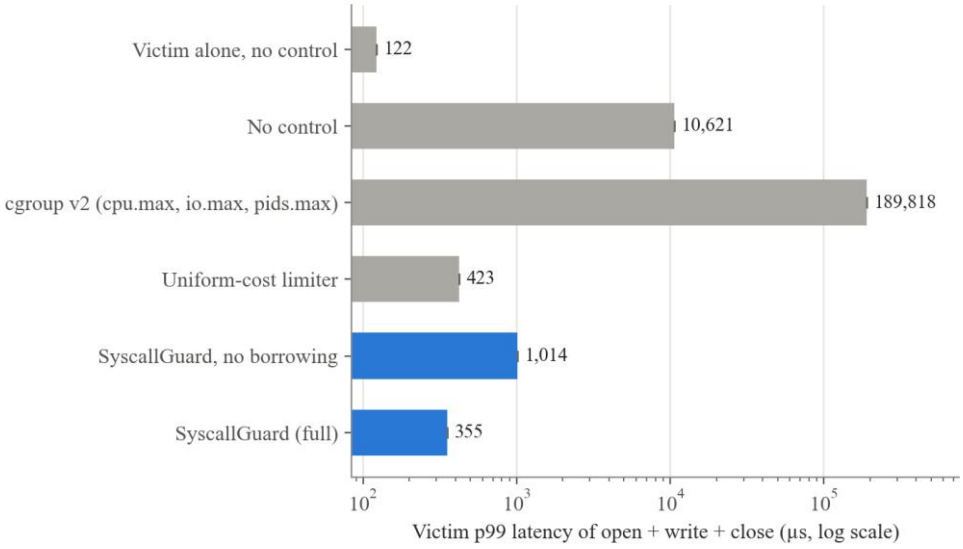


Fig. 8.1 E3: Victim 99th-Percentile Latency (log scale)

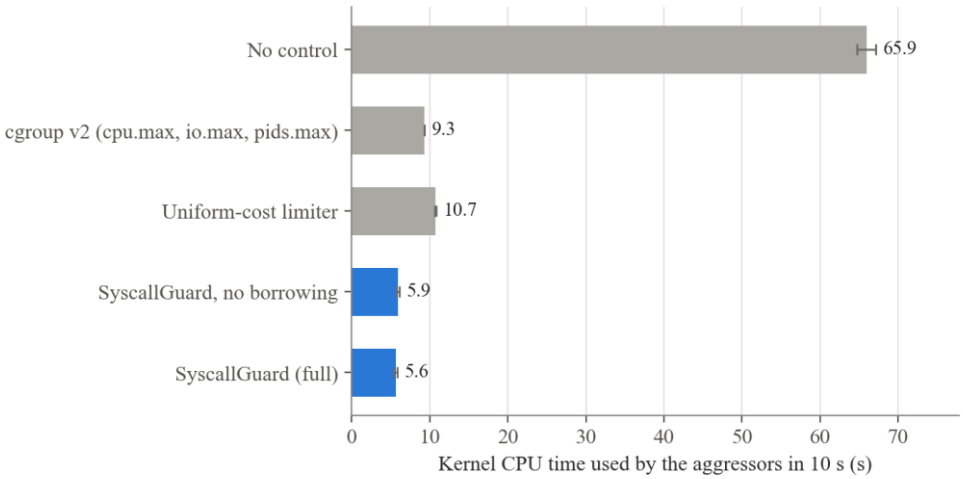


Fig. 8.2 E3: Kernel CPU Time Consumed by the Aggressors in 10 s

The neighbour hurts. Alone, the victim's p99 is 122 µs. With the aggressors uncontrolled it rises to 10,621 µs, almost 90 times worse: the metadata storm holds the shared directory's lock most of the time.

cgroup v2 makes it worse. Limiting the aggressors to one CPU raised the victim's p99 to 189,818 µs. When a group's CPU quota runs out, the kernel deschedules its threads wherever they are, including while they hold the directory lock, and the victim then waits until the group's next period. The cgroup reduced the aggressors' work, but it moved the waiting onto the victim.

SyscallGuard protects the victim. Under full SyscallGuard the victim's p99 is 355 µs, about 30 times better than with no control. sgd holds the aggressors' calls *before* they enter the kernel, so a held thread never owns a kernel lock. The aggressors' kernel CPU fell from 65.94 ± 1.18 s to 5.62 ± 0.21 s. The p99 is still above the victim's native 122 µs because the victim's own costed calls also make a round trip to sgd (Section 8.8).

Cost weighting matters. The uniform-cost limiter has the same budget and priorities and differs only in charging every call 1 unit. Because an fsync is as cheap to it as a read, it let the fsync flood run 2,737 fsyncs per second, about 100 times more than SyscallGuard (26) and even more than with no control, and the aggressors used about twice as much kernel CPU. Its victim p99 (423 ± 216) varied widely between runs. Without borrowing, SyscallGuard's victim p99 also varied strongly between runs (1,014 ± 1,018 µs), while the full configuration stayed stable (355 ± 39 µs); three runs are too few to say whether borrowing causes that difference.

8.5 E4: Fork Pressure

Table 8.5 E4: Victim Next to 16 Fork Workers

Config	Victim p99 (µs)	Forks/s	Fork EAGAIN	Forker kernel CPU (s)	sgd CPU (s)
no control	167 ± 47	71,125 ± 3,104	0	42.84 ± 0.26	–
cgroup v2	159 ± 27	3,326 ± 159	8,355 ± 437	5.71 ± 0.07	–
uniform-cost limiter	1,693 ± 91	32,107 ± 730	0	28.48 ± 0.62	1.34 ± 0.03
SyscallGuard (full)	495 ± 74	451 ± 1	0	0.97 ± 0.06	0.17 ± 0.01
SyscallGuard, fork policy reject	523 ± 65	451 ± 0	8,010 ± 41	1.18 ± 0.01	0.22 ± 0.02

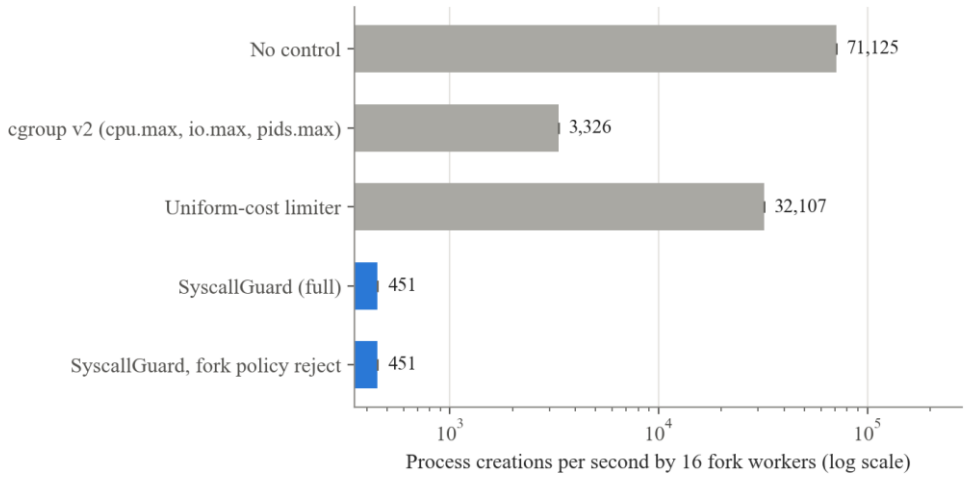


Fig. 8.3 E4: Process Creation Rate (log scale)

Uncontrolled, the 16 workers created 71,125 ± 3,104 processes per second and used 42.84 ± 0.26 s of kernel CPU in 10 s. SyscallGuard held them at about 451 forks per second, the rate their budget pays for, and their kernel CPU fell to 0.97 ± 0.06 s, about 40 times less. With the default fork policy (brief), the workers saw no EAGAIN at all: their forks waited for budget. With policy reject, the same rate was enforced by failing about 8,000 forks; the 10 ms delay before each rejection keeps those retries from busying sgd. The uniform limiter, where a fork costs the same as a read, allowed 70 times more forks, and its victim p99 was the worst of all (1,693 ± 91 µs) because sgd itself was busy with 32,000 notifications per second. The cgroup's `pids.max` and CPU limit held the forks at about 3,300 per second by failing about 8,400 of them. The

victim's p99 under SyscallGuard (495 ± 74 µs) is higher than with no control, again because the victim's own costed calls go through sgd.

8.6 E5: Borrowing

Table 8.6 E5: A Fixed Burst With and Without Borrowing

Config	Completed	Time (s)	Waited calls	Loans	Borrowed units
SyscallGuard, no borrowing	3/3	3.08 ± 0.01	295 ± 1	0	0
SyscallGuard (full)	3/3	2.58 ± 0.00	242	624 ± 8	17,322 ± 4

The burst (about 40,000 units) runs at priority 5 next to an idle priority-5 job with R_g = 20,000 units/s, so its share is 10,000 units/s with a capacity of 10,000 units. Without borrowing it needs about (40,000 − 10,000) / 10,000 = 3.0 s, and took 3.08 ± 0.01 s. Borrowing can shorten it by at most $\beta \cdot C_j = 0.5 \cdot 10{,}000$ units, which is 0.5 s at its own rate. With borrowing it took 2.58 ± 0.00 s: the full 0.5 s saving the design allows, in about 620 loans totalling 17,322 ± 4 units.

8.7 E6: Throttled Build

Table 8.7 E6: make -j4 at a Quarter of Its Demand

Fork policy	Builds completed	Build time (s)	Waited calls	Rejected calls
reject	0/3	–	124 ± 2	3
brief	3/3	3.57 ± 0.49	974 ± 60	0
wait	3/3	3.27 ± 0.00	971 ± 51	0

When a build is throttled hard, its process creations run short of budget. With policy reject, the first fork that found the budget empty failed and `make` stopped with "Resource temporarily unavailable": 0 of 3 builds completed. With the default policy brief and with wait, every build completed, in 3.57 ± 0.49 s and 3.27 ± 0.00 s against 1.16 ± 0.25 s unthrottled. Together with E4 this shows why brief is the default: it lets a throttled build finish while a fork storm, whose forks cannot all be funded within 500 ms, is still contained.

8.8 Where the Overhead Goes

E3 and E4 showed that a job's own costed calls become slower under sgd. `eval/diagnose.py` measured this for a victim running alone, and `sg_wakeup_trace.py` split each costed call into stages with eBPF tracepoints. Table 8.8 gives the victim's latency and Fig. 8.4 the median time of each stage.

Table 8.8 Victim Alone: Cost of an Occasional Costed Call

Config	Victim p50 (µs)	Victim p99 (µs)	Run-queue wait (ms)	sgd CPU (s)	sgd CPU migrations
native (no sgd)	56 ± 2	124 ± 34	6.5 ± 4.5	–	–
sgd, sync wake-up (default)	277 ± 2	512 ± 10	7.5 ± 0.5	0.08 ± 0.01	1,806 ± 5
sgd, sync wake-up off (-W)	390 ± 8	710 ± 55	5.8 ± 5.8	0.12 ± 0.01	41 ± 9
sgd, spinning (-S 200)	278 ± 7	751 ± 19	4.5 ± 2.7	0.33 ± 0.01	32 ± 20

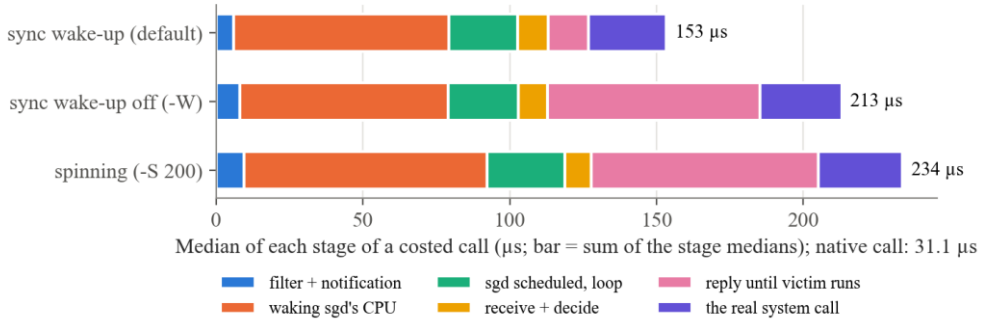


Fig. 8.4 Stages of a Costed Call of the Victim Running Alone (medians)

Natively, the victim's open + write + close round has a median of 56 µs. Under sgd with the default synchronous wake-up it is 277 µs, so each of its two costed calls (openat and write) pays roughly 100 µs more. The traced run shows where: a costed call takes 161 µs at the median instead of 31 µs natively, and the largest single stage is waking sgd (cpu_wake, 74 µs). sgd sleeps between the victim's calls, so the kernel has to wake it on another, idle CPU: 1,878 of sgd's 2,764 wake-ups were on another CPU. sgd's own work (receiving the notification and deciding) takes about 10 µs. Synchronous wake-up shortens the reply (back) from 72 µs to 14 µs because the victim is woken on the CPU sgd runs on (2,701 of 2,724 victim wake-ups on the same CPU), which is why turning it off raises the median to 390 µs. Spinning for 200 µs after each notification keeps sgd awake, but costs four times the CPU and did not lower the p99.

Table 8.9 Victim Next to 16 Fork Workers: Synchronous Wake-up On and Off

Config	Victim p50 (µs)	Victim p99 (µs)	Run-queue wait (ms)	Forks/s	sgd CPU (s)	sgd CPU migrations
uniform, sync on	358 ± 24	1,709 ± 76	294.4 ± 16.6	33,097 ± 68	1.31 ± 0.03	69,383 ± 558
uniform, sync off	176 ± 8	615 ± 113	103.0 ± 22.9	44,444 ± 96	3.84 ± 0.03	4,245 ± 278
SyscallGuard, sync on	226 ± 24	466 ± 6	6.7 ± 0.5	452 ± 0	0.15 ± 0.01	2,533 ± 55
SyscallGuard, sync off	372 ± 11	703 ± 25	8.5 ± 7.6	453 ± 0	0.28 ± 0.02	517 ± 22

Synchronous wake-up is not always a gain. Under the uniform limiter, where sgd handles over 33,000 fork notifications per second, it raised the victim's p99 from 615 µs to 1,709 µs and made sgd migrate between CPUs 69,383 times in 10 s instead of 4,245: each synchronous wake-up pulls sgd to the CPU of the thread that notified it. sgd waits in epoll, which does not pass the same-CPU hint on, so under heavy fan-in it is dragged from CPU to CPU. With SyscallGuard's calibrated costs, the fork workers are held to about 450 forks per second, the notification load stays low, and synchronous wake-up helps (p99 466 µs against 703 µs without it). Choosing synchronous wake-up per job or adaptively, and waiting in a way that keeps the same-CPU hint, are left as future work.

8.9 Discussion

- **Control before the kernel beats control inside it.** The biggest difference between SyscallGuard and cgroup v2 in E3 is not how much the aggressors were limited but *where* they were stopped. A thread stopped by a CPU quota can be holding a lock; a call held by sgd has not started.
- **What a call costs matters more than how many calls are made.** With identical budgets and priorities, charging calls by their measured cost cut the fsync rate about 100 times and the aggressors' kernel CPU about 2 times compared with counting calls, and held fork workers 70 times lower.
- **Holding is better than failing, within limits.** Waiting kept every job in E3–E5 free of errors; the fork policies show where a bounded wait followed by EAGAIN is the better choice.

- **Borrowing does what it promises and no more.** It saved exactly the 0.5 s bound in E5 and never created tokens, so the global budget was never exceeded.
- **The price is the round trip.** Calls within budget are not slowed in throughput (E2), but each costed call of a lightly loaded job pays a few hundred microseconds, most of it waking sgd on another CPU (Section 8.8).

8.10 Advantages and Limitations

Table 8.10 SyscallGuard Compared with the Baselines

Aspect	cgroup v2	Uniform-cost limiter	SyscallGuard
What is limited	CPU time, I/O bandwidth/IOPS, process count	Number of costed calls	Kernel work, as calibrated cost per call
Where a job is stopped	Anywhere, including inside the kernel holding locks	Before the call enters the kernel	Before the call enters the kernel
Victim p99 in E3 (µs)	189,818 ± 870	423 ± 216	355 ± 39
fsync flood in E3 (ops/s)	88 ± 7	2,737 ± 740	26 ± 0
Fork storm in E4 (forks/s)	3,326 ± 159	32,107 ± 730	451 ± 1
Over-limit behaviour	Throttle or fail	Wait	Wait, bounded wait or fail, per class and job
Unused budget	Not shared across groups	Not shared	Lent to bursty jobs through the loan ledger
Overhead per governed call	None	Round trip to sgd	Fast path ~4 ns; round trip ~18 µs
Needs	Root (or delegation)	Nothing special	Linux ≥ 5.5; no root, no kernel module

The current implementation also has these limitations:

- **Round-trip latency.** An occasional costed call of a lightly loaded job pays a few hundred microseconds, mostly for waking sgd on an idle CPU. Synchronous wake-up helps a lightly loaded job but, under heavy notification load, it drags sgd from CPU to CPU (Table 8.9).
- **Single-threaded supervisor.** One sgd thread handles every notification; very high rates of costed calls from many jobs are bounded by it.

- **Uncharged paths.** `sendmsg()` and `recvmsg()` are not charged, because the launcher needs `sendmsg()` before sgd holds the listener; socket I/O through them bypasses the data budget.
- **Charge before the call.** A call is charged when admitted, even if it then fails.
- **Machine-specific costs.** A calibration file is valid only for the machine and file system it was measured on.
- **Lifecycle.** A filter cannot be removed from a running process; if sgd exits, costed calls of running jobs fail with ENOSYS. A job ends only when all its processes have been reaped.
- **Evaluation environment.** All measurements were made under WSL2 on one laptop; bare-metal and multi-socket results may differ, particularly for wake-up costs.

CHAPTER 9

CONCLUSION

The project delivers a working system-call governor for Linux. SyscallGuard intercepts the system calls of each job and its whole process tree with seccomp-BPF and seccomp user notification, charges each call by its kernel cost as measured with eBPF on the target machine, and enforces priority-proportional budgets with token buckets, a wait queue with aging, borrowing through a loan ledger, and per-job fork policies. It runs on an unmodified kernel without root, is tested by unit and integration tests, and can be observed and controlled at run time with `sgctl`, Prometheus metrics and a JSON event log.

The evaluation shows that controlling system calls by their cost, before they enter the kernel, protects a latency-sensitive job where existing mechanisms do not. Next to a metadata storm and an fsync flood, SyscallGuard reduced the victim's 99th-percentile latency from 10,621 µs to 355 µs, while a cgroup v2 CPU limit made it worse (189,818 µs) by descheduling lock holders. Charging calls by cost instead of counting them cut the admitted fsync rate about 100 times. A fork storm was held to the rate its budget pays for, a throttled build still completed under the default fork policy, and borrowing shortened a burst by exactly its theoretical bound. Calls that are not costed pay about 4 ns, and an unthrottled build was not measurably slower.

The main cost is the round trip of a costed call to the supervisor, which for a lightly loaded job is dominated by waking the supervisor on another CPU. Future work follows from the limitations:

- decide per job, or adaptively under load, whether to request synchronous wake-up, and let sgd wait with a mechanism that keeps the same-CPU wake-up hint (for example `ppoll` on the listeners instead of epoll);
- split sgd into several threads, each owning a subset of the jobs, with the global bucket shared between them;
- charge `sendmsg()`/`recvmsg()` once the listener has been handed over, and refund the cost of calls that fail;
- integrate with cgroups, so that a container or systemd service can be governed as one job, and repeat the evaluation on bare metal and on larger machines.

REFERENCES

[1] Linux man-pages project. seccomp(2): operate on Secure Computing state of the process. https://man7.org/linux/man-pages/man2/seccomp.2.html

[2] Linux man-pages project. seccomp_unotify(2): seccomp user-space notification mechanism. https://man7.org/linux/man-pages/man2/seccomp_unotify.2.html

[3] The Linux Kernel Documentation. Seccomp BPF (SECure COMPuting with filters). https://docs.kernel.org/userspace-api/seccomp_filter.html

[4] McCanne, S. and Jacobson, V. (1993). The BSD Packet Filter: A New Architecture for User-level Packet Capture. Proceedings of the USENIX Winter 1993 Conference, pp. 259–269.

[5] Brauner, C. (2020). The Seccomp Notifier – New Frontiers in Unprivileged Container Development. https://brauner.io/2020/07/23/seccomp-notify.html

[6] The Linux Kernel Documentation. Control Group v2. https://docs.kernel.org/admin-guide/cgroup-v2.html

[7] Silberschatz, A., Galvin, P. B. and Gagne, G. (2018). Operating System Concepts, 10th ed. Wiley.

[8] Tanenbaum, A. S. and Bos, H. (2014). Modern Operating Systems, 4th ed. Pearson.

[9] Tanenbaum, A. S. and Wetherall, D. J. (2011). Computer Networks, 5th ed. Pearson (Section 5.4: traffic shaping, leaky and token buckets).

[10] Jain, R., Chiu, D.-M. and Hawe, W. (1984). A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems. DEC Research Report TR-301.

[11] Gregg, B. (2019). BPF Performance Tools: Linux System and Application Observability. Addison-Wesley.

[12] bpftrace: High-level tracing language for Linux. https://github.com/bpftrace/bpftrace

[13] Linux man-pages project. unix(7) (SCM_RIGHTS), epoll(7), timerfd_create(2), signalfd(2), clone(2). https://man7.org/linux/man-pages/

[14] Prometheus. Exposition formats. https://prometheus.io/docs/instrumenting/exposition_formats/

APPENDIX A

A.1 How to Run the Project

Requirements: Linux 5.5 or later (5.19+ recommended, 6.6+ for synchronous wake-up), `gcc`, `make` and `python3`; `bpftrace` and root only for calibration and diagnostics.

```
sudo apt install build-essential python3 bpftrace
make                    # builds sgd, sg-launch, sgctl, step1_hello
make test               # all tests (no root)

./build/sgd -r 3000 -l events.jsonl &              # global budget 3000 units/s
./build/sg-launch -p 8 -n build  -- make -j4        # high priority
./build/sg-launch -p 2 -n logger -- ./noisy-logger   # low priority
./build/sg-launch -r 500 -- ./batch                  # fixed 500 units/s
./build/sg-launch -f reject -- ./untrusted           # fork fails when short
./build/sgctl status                                 # also: loans, metrics, set-prio
python3 tools/sg_analyze.py events.jsonl             # metrics from the log

make calibrate          # root: writes costs/$(uname -r).conf
./build/sgd -c costs/$(uname -r).conf ...
make eval-bins
sudo python3 eval/run_eval.py --label myhost      # about 15 minutes
sudo python3 eval/diagnose.py --label myhost      # about 7 minutes
```

A.2 sgd Options

Table A.1 sgd Command-Line Options

Option	Meaning	Default
`-r rate`	Global budget R_g, cost units per second	200000
`-H ms`	Burst horizon H (bucket capacity = rate × H)	1000
`-T ms`	T_{max}: a call waiting longer is rejected with EAGAIN	2000
`-F ms`	Longest wait for process creation under fork policy brief	500
`-a ms`	Aging quantum q	500
`-q n`	Waiting calls allowed per job	256
`-I ms`	Idle window after which a job lends to any priority	1000
`-B`	Disable borrowing	on
`-k pct`	Minimum loan as % of the borrower's capacity (0 = exact deficit)	10
`-R ms`	Delay before a budget rejection is answered (0 = at once)	10
`-S us`	Spin after a notification (turns synchronous wake-up off)	0
`-W`	Do not request synchronous wake-up	requested
`-c file` / `-L`	Load calibrated costs / print the cost table and exit	seed costs
`-m port`	Serve Prometheus metrics on 127.0.0.1:port	off
`-l file`	JSON-lines event log	off

Option	Meaning	Default
`-s path`	Control socket	/tmp/syscallguard-<uid>.sock

A.3 sg-launch Options

Table A.2 sg-launch Command-Line Options

Option	Meaning	Default
`-p prio`	Priority 1 (lowest) .. 10 (highest)	5
`-r rate`	Fixed budget in units/s instead of a priority share	share
`-n name`	Job name shown by sgd and sgctl	command name
`-f policy`	Fork policy: brief, wait or reject	brief
`-s sock`	sgd socket	/tmp/syscallguard-<uid>.sock

Exit status: the command's own status; 125 if sg-launch itself failed, 126 if the command could not be executed, 127 if it was not found.